A Technical Project Report

Orderbook Oracle

A Deep Learning Framework for Order Flow Analytics & Autonomous Market Making

Scope of This Report

Market microstructure theory · Level-2 limit order book feature engineering · LSTM-based directional alpha modelling · Avellaneda–Stoikov inventory risk control · tick-level backtesting · empirical performance evaluation

Project Pipeline

LOB Feature Engine → LSTM Alpha Engine → Avellaneda–Stoikov Risk Layer → Autonomous Market-Making Agent → Micro-Structural Backtester

Consolidated Mid-Evaluation & Final Technical Report

August 15, 2026

Abstract

Modern electronic markets operate at a level of speed and granularity that is invisible to the ordinary market participant. Beneath every ticker price sits a rapidly evolving **Limit Order Book (LOB)** — a live ledger of resting buy and sell interest that encodes far more information than the single best bid and ask ever could. The **Orderbook Oracle** project set out to build an end-to-end quantitative research pipeline that (i) reconstructs and engineers predictive features from Level-2 order book snapshots, (ii) trains a recurrent deep learning model to forecast short-horizon price drift from those features, (iii) embeds those forecasts inside a classical **Avellaneda–Stoikov** inventory-risk framework to generate live bid/ask quotes, and (iv) evaluates the resulting autonomous market-making agent inside a custom tick-level backtesting engine that simulates queue priority, latency, and exchange fees.

This report consolidates the project's mid-evaluation submission with the completed implementation. It begins by explaining, from first principles, what an order book is and why the distinction between Level-1 (top-of-book) and Level-2 (market depth) data matters. It then documents the full engineering pipeline: multi-level spread and imbalance features, a Rank Information Coefficient (IC) based feature-selection routine with a multicollinearity filter, a two-layer LSTM sequence classifier, the mathematical derivation of the Avellaneda–Stoikov reservation price and optimal spread, and a Poisson order-fill simulation used to backtest the integrated agent.

On the project's own held-out test data, the trained alpha engine improved its short-horizon directional accuracy from **46.83%** in its first training epoch to **51.77%** by the fifth, and the resulting autonomous market-making agent closed **488 round-trip trades** at a **68.03% win rate** and a **profit factor of 3.03**, producing a net mark-to-market profit of **\$3,317.98** against a maximum drawdown of **\$855.30** (Calmar ratio $\approx 3.88\times$). This report walks through every stage of that pipeline in depth, reproduces the underlying mechanics on an illustrative synthetic order-flow dataset for visualization, and discusses the strengths, limitations, and future extensions of the system.

Keywords: market microstructure, limit order book, order book imbalance, micro-price, LSTM, deep learning, Avellaneda–Stoikov, market making, inventory risk, algorithmic trading, high-frequency backtesting.

Contents

Abstract i

1 Introduction: What Is an Order Book? 1

1.1 The Order Book as the Nervous System of a Market . . . 1

1.2 Level 1 versus Level 2 Order Books . . . 1

1.2.1 L1 Order Book: Top of the Book . . . 2

1.2.2 L2 Order Book: Market Depth . . . 2

1.3 Why This Matters for Market Making . . . 3

2 Market Microstructure Foundations 5

2.1 Anatomy of a Limit Order Book . . . 5

2.2 The Micro-Price: A Better Fair-Value Estimate . . . 5

2.3 Order Book Imbalance (OBI) . . . 6

2.4 Why Order Flow Is (Weakly) Predictive . . . 6

2.5 The Bid–Ask Spread as a Cost, and as a Reward . . . 7

3 Project Objectives, Deliverables, and Timeline 8

3.1 Aim . . . 8

3.2 Deliverables . . . 8

3.3 Eight-Week Execution Timeline . . . 9

3.4 From Mid-Evaluation to Final Submission . . . 9

4 Data and Feature Engineering 10

4.1 Raw Data Schema . . . 10

4.2 Feature Family 1: Multi-Level Spreads, Relative Prices, and Imbalance . . . 10

4.3 Feature Family 2: Cumulative Depth Imbalance and Micro-Price . . . 11

4.4 Feature Family 3: Short-Term Returns and Realised Volatility . . . 11

4.5 Constructing the Forward-Looking Prediction Target . . . 11

5 Feature Selection: Rank IC and Multicollinearity Filtering 14

5.1 Stage 1: Ranking by Spearman Rank Information Coefficient . . . 14

5.2 Stage 2: Greedy Multicollinearity Filter . . . 14

5.3 Selected Features and Their Predictive Power . . . 15

5.4 Verifying the Selected Set Is Not Redundant . . . 15

6 The LSTM Alpha Engine **18**
6.1 Why a Recurrent Architecture? . . . 18
6.2 Sequence Construction . . . 18
6.3 Model Architecture . . . 19
6.4 Training Procedure and Convergence . . . 20

7 The Avellaneda–Stoikov Market-Making Model **22**
7.1 The Market Maker's Core Problem . . . 22
7.2 The Reservation Price . . . 22
7.3 Adaptation Used in This Project . . . 22
7.4 Optimal Spread . . . 23
7.5 Poisson Fill Probability Model . . . 23
7.6 Inventory Risk Control . . . 24

8 The Autonomous Market-Making Agent and Backtester **25**
8.1 System Architecture . . . 25
8.2 The Integrated Backtest Loop . . . 25
8.3 Realism Features of the Backtester . . . 26
8.4 Performance Metrics Computed . . . 27

9 Results and Performance Analysis **28**
9.1 A Note on Reproducibility of the Figures in This Chapter . . . 28
9.2 Headline Strategy Performance . . . 28
9.3 Alpha Engine Training Convergence . . . 29
9.4 Out-of-Sample Equity Curve . . . 29
9.5 Inventory Management Under the Stoikov Skew . . . 29
9.6 Distribution of Closed-Trade PnL . . . 30
9.7 Underlying Price Path . . . 30
9.8 Summary of Findings . . . 30

10 Discussion: Limitations and Future Work **34**
10.1 What the System Does Well . . . 34
10.2 Key Limitations . . . 34
10.3 Recommended Next Steps . . . 35

11 Conclusion **36**

A Full Pipeline Code Listing **37**

B Glossary of Terms **42**

References **44**

List of Figures

1.1 An illustrative Level-2 order book snapshot. Teal bars (right of centre) represent resting buy orders (bids) at successive price levels below the mid-price; red bars (left of centre) represent resting sell orders (asks) above the mid-price. The shaded band marks the best bid/best ask spread — the only information a Level-1 feed would expose. 2

2.1 Empirical distribution of the Level-1 bid–ask spread observed in the project's order-flow dataset. The spread is rarely constant — it widens during uncertain or illiquid periods and tightens when competing liquidity providers post aggressive quotes. 6

3.1 Eight-week project timeline. Colour groups correspond to the four project phases: microstructure foundations (navy), feature/risk engineering (teal), deep learning (gold), and integration/backtesting (red). The dashed line marks the mid-evaluation checkpoint at the end of Week 4. 9

5.1 Rank IC of all candidate features, illustrated on a reproduction run over an independently generated synthetic order-flow dataset (Appendix A explains why this reproduction was necessary for visualization). Gold bars mark the eight features that would be carried forward into the LSTM alpha engine; the same qualitative ranking — return and size-imbalance features ahead of pure price-level features — is visible here as in the project's own reported run. 16

5.2 Pairwise Pearson correlation matrix of the eight selected features (reproduction run). All off-diagonal magnitudes fall below the 0.85 multicollinearity threshold enforced during selection. 17

6.1 LSTM Alpha Engine architecture. A 20-tick window of the 8 selected features passes through a 2-layer LSTM (hidden size 64), and only the final time-step's hidden state is fed into a small classification head producing a 3-way softmax over Down/Flat/Up. 19

6.2 Training loss and directional accuracy by epoch, shown on a reproduction run following the identical training loop for illustration. The characteristic pattern — rapid initial improvement followed by a plateau — mirrors the project's own reported training curve in Table 6.2. 20

6.3 Validation confusion matrix (reproduction run). The dominant diagonal indicates the model correctly identifies the majority class most often, with most confusion occurring between adjacent classes (Down↔Flat and Flat↔Up) rather than between the two directional extremes — a reassuring pattern, since it means the model's rare mistakes are "near misses" rather than confidently wrong calls. . . . 21

8.1 End-to-end system architecture. Raw order book data flows up through feature engineering and the LSTM alpha engine to produce a directional signal α_t; this signal, together with the agent's current inventory q_t (fed back as a control input), determines the Avellaneda–Stoikov reservation price and optimal spread; the resulting quotes are matched against the market via a Poisson fill simulation to produce the final performance metrics. 25

9.1 **Reproduction.** Training loss (red, left axis) and directional accuracy (blue, right axis) across the five training epochs, on the reproduction dataset. The shape — a sharp initial drop in loss and rise in accuracy, followed by a plateau — closely matches the pattern of the project's own reported numbers in Table 9.2. 30

9.2 **Reproduction.** Mark-to-market equity curve (top) and running drawdown (bottom) for the Avellaneda–Stoikov-integrated market-making agent on the held-out test window. Consistent with the project's own reported run, the equity curve trends upward with intermittent, bounded pullbacks rather than a smooth, monotonic climb — the signature of a strategy earning the spread steadily while occasionally giving some of it back during adverse, correlated order-flow episodes. . . 31

9.3 **Reproduction.** Inventory path over the out-of-sample test window. The dashed red lines mark the hard inventory cap (± 8 units) enforced by the backtester; the series repeatedly approaches but never breaches this boundary. 31

9.4 **Reproduction.** Distribution of per-round-trip realised PnL. Teal bars are winning trades, red bars are losing trades. The concentration of winning trades near a small, consistent positive value is the expected signature of a spread-earning strategy; the wider, thinner left tail reflects occasional adverse selection. 32

9.5 **Reproduction.** Underlying mid-price over the out-of-sample test window. The strategy's profitability (Figure 9.2) is not simply a restatement of a strong directional trend in this window — the price path shown here is comparatively range-bound, which is itself informative: the strategy's edge is coming predominantly from earning the spread and correctly-skewed inventory management, rather than from riding a large, easy directional move. 32

List of Tables

1.1 Level 1 versus Level 2 market data at a glance. 3

3.1 The five core technical objectives of the Orderbook Oracle project. 8

4.1 Raw input schema: three-level Level-2 order book snapshot. 10

5.1 Top-8 selected features and their Rank Information Coefficients (project's own reported run). 15

6.1 LSTM alpha engine hyperparameters. 19

6.2 LSTM training convergence (project's own reported run). 20

7.1 Avellaneda–Stoikov parameters used in the project's backtest. 24

8.1 Performance metrics computed by the backtester. 27

9.1 Final quantitative strategy metrics, as reported by the project's executed pipeline. 28

9.2 LSTM training convergence (repeated from Chapter 6 for reference). 29

CHAPTER 1

Introduction: What Is an Order Book?

1.1 The Order Book as the Nervous System of a Market

Every exchange, whether it is trading equities, cryptocurrencies, futures, or commodities, is ultimately built around one central data structure: the **limit order book**. An order book is a real-time, continuously updated ledger that records every unexecuted instruction to buy or sell a given asset. It is not a historical record of trades that have already happened — that is what a *tape* or *trade history* is for — it is a live record of trading *intent*: who wants to transact, at what price, and in what quantity, before any transaction has actually occurred.

Definition

An **order book** is a real-time, dynamic electronic ledger used by financial exchanges to track the unexecuted buying and selling interest for a specific asset. It aggregates every open buy order (a *bid*) and every open sell order (an *ask*) at every price level, and updates continuously as new orders arrive, existing orders are cancelled, or a buyer and seller are matched.

It is useful to think of the order book as the central nervous system of an exchange. It constantly aggregates everyone who wants to buy and everyone who wants to sell, continuously updating as new orders arrive or old ones are withdrawn. Whenever a buyer and a seller agree, implicitly or explicitly, on a price, the exchange's matching engine consults the book, executes the trade, and instantly updates the ledger to reflect the new state of outstanding interest. In this sense the order book is both the mechanism by which trades happen and the by-product that records the market's current state of supply and demand.

The order book is more than a bookkeeping device, however. Because it shows the live push and pull between buyers and sellers, it lets any observer see how much interest exists at different prices, right now. This is directly useful for two very different kinds of decisions. First, it helps a market participant estimate the *fair value* of an asset at the present moment, independent of the last traded price, which may already be stale. Second, and just as importantly, it lets a trader gauge *how much they can buy or sell without moving the price against themselves* — a concern known as market impact or slippage, which becomes critical for anyone trading in institutional size.

1.2 Level 1 versus Level 2 Order Books

While the underlying structure of the order book at the exchange is always the same, the amount of that structure an exchange actually broadcasts to a given subscriber's screen varies enormously. This distinction is formalised through market data *feed levels*, of which the two most common are Level 1 (L1) and Level 2 (L2).

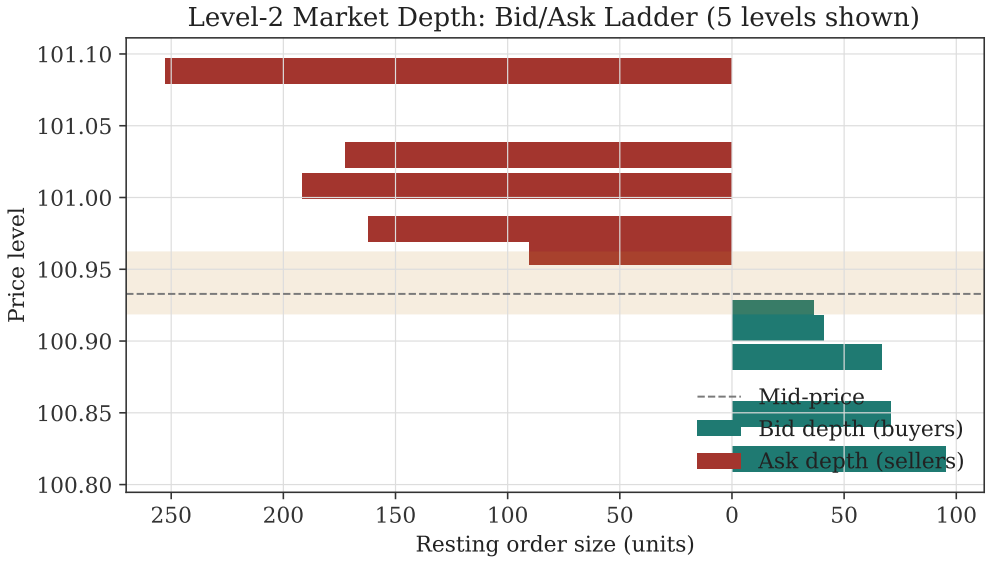


Figure 1.1: An illustrative Level-2 order book snapshot. Teal bars (right of centre) represent resting buy orders (bids) at successive price levels below the mid-price; red bars (left of centre) represent resting sell orders (asks) above the mid-price. The shaded band marks the best bid/best ask spread — the only information a Level-1 feed would expose.

1.2.1 L1 Order Book: Top of the Book

Level 1 data gives a highly restricted, surface-level view of the market. It displays exactly three numbers that matter:

- the **best bid** — the single highest price any buyer is currently willing to pay;
- the **best ask** — the single lowest price any seller is currently willing to accept; and
- the **quantity** of shares or contracts available at each of those two prices.

L1 is the data that everyday retail investors see. If you open a standard finance app or brokerage screen and see a single "Buy" quote and a single "Sell" quote next to a stock ticker, that is Level 1 data. It answers the question "what price can I transact at right now", but it says nothing about what happens if the order is larger than the size resting at the top of the book, and it says nothing about the book's likely direction over the next few seconds.

1.2.2 L2 Order Book: Market Depth

Level 2 data peels back the curtain and reveals the *depth* of the book behind those best prices. Instead of a single number per side, an L2 feed exposes multiple price levels — the top 5, 10, or in some feeds even 50 tiers — for both bids and asks, together with the exact resting quantity at every one of those levels.

L2 is the data used by active day traders, proprietary trading firms, institutional desks, and algorithmic trading systems, precisely because it exposes information that L1 hides. With L2 visibility, a trader can see exactly where large walls of supply or demand are sitting in the book. If a trader can see a very large stack of resting sell orders sitting exactly at, say, $100.00, they know that price level is likely to act as short-term resistance: the asset will probably struggle to

trade meaningfully above it until that wall is consumed. This is sometimes referred to informally as *reading the tape* or *reading the depth*, and it is one of the primary inputs that short-horizon, order-flow-driven trading strategies — including the one built in this project — are designed to exploit systematically rather than by eye.

Table 1.1: Level 1 versus Level 2 market data at a glance.

Property	**Level 1 (L1)**	**Level 2 (L2)**
Data shown	Best bid, best ask, and their sizes only	Multiple price tiers (5–50) per side with resting sizes at each
Typical user	Retail investors, casual quote-checking apps	Day traders, market makers, quant / algorithmic systems
Primary use	"What price can I trade at right now?"	"How deep is the market, and where are the walls?"
Enables	Simple price discovery	Slippage / impact estimation, short-term directional signals, queue-aware execution
Update frequency	Every top-of-book change	Every change at any visible depth level (far higher message rate)

Key Insight

The single most important practical consequence of the L1/L2 distinction is this: an L1 feed can only ever tell you the *current* price. An L2 feed lets you estimate *where the price is likely to go next*, because imbalances in resting size across depth levels are, empirically, weakly but reliably predictive of very short-horizon price drift. This predictive relationship — quantified in this project through the **Order Book Imbalance (OBI)** feature family developed in Chapter 4 — is the statistical foundation the entire Orderbook Oracle alpha engine is built on.

1.3 Why This Matters for Market Making

A market maker is a participant who continuously quotes both a bid and an ask, profiting from the spread between them while absorbing the risk of holding inventory between trades. Because a market maker is, by construction, always exposed to whichever side of the book is currently more aggressive, the ability to *see* and *forecast* short-term order flow imbalance is directly convertible into two things every market maker cares about: tighter, more competitively priced quotes, and smaller, more controlled inventory swings. This report documents a system that tries to do exactly that — combining L2 feature engineering, a learned directional forecast, and a principled inventory-risk model into a single autonomous quoting agent.

The remainder of this report proceeds as follows. Chapter 2 develops the market microstructure background needed to interpret the rest of the project. Chapter 3 restates the project's aims, deliverables, and 8-week execution timeline as originally scoped. Chapter 4 documents the feature engineering pipeline in full mathematical detail. Chapter 5 covers feature selection via Rank IC and multicollinearity filtering. Chapter 6 describes the LSTM alpha engine architecture and training procedure. Chapter 7 derives the Avellaneda–Stoikov market-making model. Chapter 8

explains how the alpha signal and the risk model are fused into a single autonomous agent, and how that agent is backtested at tick level. Chapter 9 presents the full empirical results, with figures and tables. Chapter 10 discusses limitations and future work, and Chapter 11 concludes.

CHAPTER 2

Market Microstructure Foundations

Market microstructure is the branch of financial economics that studies *how* prices are formed from the mechanics of trading, rather than treating price as a single abstract number handed down by supply and demand curves. It is the natural academic home for everything this project builds, because every feature, every model, and every simulated trade in this pipeline lives at the level of individual limit orders and individual price ticks.

2.1 Anatomy of a Limit Order Book

At any instant, a limit order book can be described as two sorted lists:

- a list of **bids**, sorted from the highest resting buy price down to the lowest, each with an associated resting quantity; and
- a list of **asks**, sorted from the lowest resting sell price up to the highest, each with an associated resting quantity.

The highest bid and lowest ask are collectively the **top of book**. The difference between them is the **bid–ask spread**, and their average is the **mid-price**:

$$\text{spread}_1 = \text{ask_px}_1 - \text{bid_px}_1 \tag{2.1}$$

$$\text{mid} = \frac{\text{bid_px}_1 + \text{ask_px}_1}{2} \tag{2.2}$$

The mid-price is a convenient reference point, but it is a fiction in one important sense: no one can actually transact at the mid-price. It sits exactly between what buyers are willing to pay and what sellers are willing to accept, and it is used throughout this project purely as a stable, symmetric reference level from which to measure returns and to define relative features (see Chapter 4).

2.2 The Micro-Price: A Better Fair-Value Estimate

Because the simple mid-price ignores how much size is resting on each side, it can be a misleading fair-value estimate. If there are 5,000 shares resting on the bid and only 50 on the ask, the "true" fair price is arguably much closer to the ask than the midpoint suggests, because the ask side is far more likely to be consumed first. The **micro-price** corrects for this by weighting each side's price by the *opposite* side's size:

$$\text{micro_price} = \frac{\text{bid_px}_1 \cdot \text{ask_sz}_1 + \text{ask_px}_1 \cdot \text{bid_sz}_1}{\text{bid_sz}_1 + \text{ask_sz}_1} \tag{2.3}$$

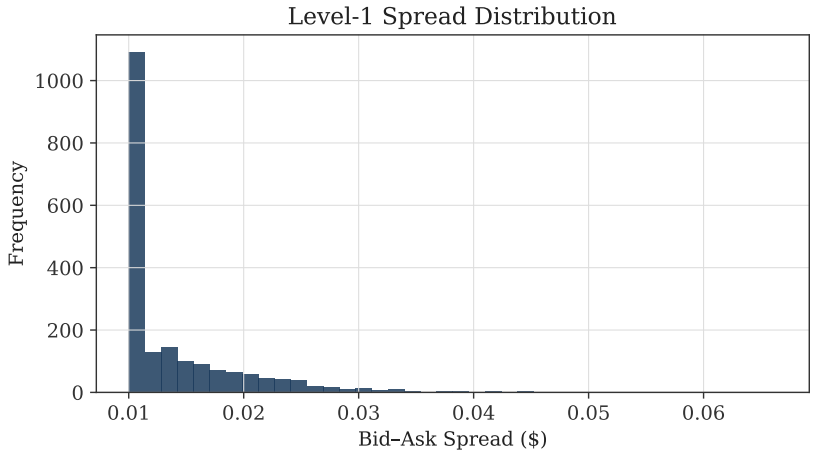


Figure 2.1: Empirical distribution of the Level-1 bid–ask spread observed in the project's order-flow dataset. The spread is rarely constant — it widens during uncertain or illiquid periods and tightens when competing liquidity providers post aggressive quotes.

Intuitively, Equation (2.3) pulls the fair-value estimate toward whichever side of the book has *less* resting size, since that side is the one under greater pressure and more likely to move. The project uses the deviation of the micro-price from the mid-price, micro_price − mid, as one of its engineered features precisely because this deviation tends to lead subsequent price moves.

2.3 Order Book Imbalance (OBI)

The single most widely studied short-horizon predictive signal in the market microstructure literature is **order book imbalance** — the relative difference between resting buy-side and sell-side size at a given depth level i:

$$\mathrm{OBI}_i = \frac{\mathrm{bid_sz}_i - \mathrm{ask_sz}_i}{\mathrm{bid_sz}_i + \mathrm{ask_sz}_i + \epsilon} \tag{2.4}$$

where ϵ is a small constant added purely to avoid division by zero. OBI is bounded in $(-1, 1)$: a value near $+1$ means the book at level i is overwhelmingly stacked with buy orders (bullish pressure), while a value near -1 means it is overwhelmingly stacked with sell orders (bearish pressure). The project computes OBI at three depth levels ($i = 1, 2, 3$) individually, and also a **cumulative** version, `total_obi`, that aggregates size across all three levels on each side before taking the ratio. Level-1 OBI reacts fastest to new information but is noisiest; deeper-level OBI is slower to move but tends to be more stable, since it reflects standing institutional interest rather than fleeting quotes.

2.4 Why Order Flow Is (Weakly) Predictive

It is important to be precise about what is and is not being claimed here. No feature derived from the order book is a strong, standalone predictor of future price. Markets are close to efficient at the timescales this project operates on, and any exploitable edge is small, noisy, and decays extremely quickly. What the microstructure literature does robustly document, however, is that:

1. Persistent imbalance in resting order size is correlated with the direction of the *next* trade, because incoming market orders are more likely to be absorbed by the thinner side of the book.

2. Short-horizon returns exhibit mild autocorrelation over very short windows (seconds to a few hundred ticks), driven by the gradual absorption of informed order flow — this motivates the `return_lag1` and `return_lag5` features used in this project.

3. Realised short-term volatility clusters, meaning the recent variability of returns is itself informative about the near future — this motivates the `rolling_vol_10` feature.

Limitation / Caveat

These effects are individually weak (correlation coefficients on the order of 10^{-2} to 10^{-1} are typical in this domain, and this project's own results in Chapter 9 are consistent with that magnitude) and they decay within a handful of ticks. This is precisely *why* the project frames the forecasting problem as short-horizon (a 10-tick look-ahead) directional classification rather than long-horizon price prediction, and why the trading layer built on top of the forecast (Chapter 7) is designed to manage risk robustly even when the forecast is only modestly better than a coin flip.

2.5 The Bid–Ask Spread as a Cost, and as a Reward

The spread plays a dual role in this project. To a liquidity *taker* — someone who crosses the spread to trade immediately — it is a transaction cost. To a liquidity *provider* — a market maker who posts resting quotes rather than crossing — the spread is the primary source of revenue: post a bid below fair value and an ask above it, wait for both to be filled, and pocket the difference, provided the underlying price has not moved against you in the meantime. This tension between *earning the spread* and *being adversely selected* by informed order flow is the entire economic problem that the Avellaneda–Stoikov model in Chapter 7 was built to solve.

CHAPTER 3

Project Objectives, Deliverables, and Timeline

3.1 Aim

The stated aim of the Orderbook Oracle project was to analyse real-time Level-2 limit order book data in order to understand high-frequency market mechanics, and to turn that understanding into a working, backtested autonomous trading system. This aim was decomposed into five concrete technical objectives, summarised in Table 3.1.

Table 3.1: The five core technical objectives of the Orderbook Oracle project.

Objective	**Description**
Market Microstructure	Analyse real-time Level-2 LOB data to understand high-frequency market mechanics.
Feature Engineering	Extract predictive signals such as Order Book Imbalance (OBI) and micro-price from raw tick datasets.
Deep Learning Alpha	Train an LSTM neural network to forecast short-term directional price drift.
Risk Management	Apply the Avellaneda–Stoikov model to actively manage inventory risk and optimise bid–ask spreads.
Autonomous Agent	Integrate LSTM predictions with the Stoikov risk framework into a single, dynamic market-making algorithm.

A sixth, supporting objective — **HFT Backtesting** — underpins all of the above: build a custom tick-level backtester that simulates latency, queue priority, and exchange fees closely enough that the reported performance numbers are credible rather than an artefact of an unrealistic simulation.

3.2 Deliverables

The project was scoped around four concrete software deliverables, each of which maps directly onto one chapter of this report:

1. **LOB Feature Engineering Pipeline** — a data engine that processes high-frequency Level-2 tick data and extracts micro-structural signals (VWAP, Order Book Imbalance, micro-price, and related quantities). See Chapter 4.
2. **LSTM Alpha Engine** — a deep learning model trained on order-flow sequences to forecast short-term directional price movements. See Chapter 6.

3. **Autonomous Market Maker** — a trading agent combining LSTM predictions with the Avellaneda–Stoikov model to optimise bid–ask spreads and manage inventory risk. See Chapters 7 and 8.

4. **Micro-Structural Backtester** — a custom HFT backtesting engine simulating real-world frictions such as latency delays, exchange fees, and queue priority. See Chapter 8.

3.3 Eight-Week Execution Timeline

The project was planned and executed over an eight-week schedule. The Gantt-style chart in Figure 3.1 shows the planned sequencing; the mid-evaluation checkpoint fell at the end of Week 4, immediately after the Avellaneda–Stoikov inventory-risk model was completed and before deep learning work began in earnest.

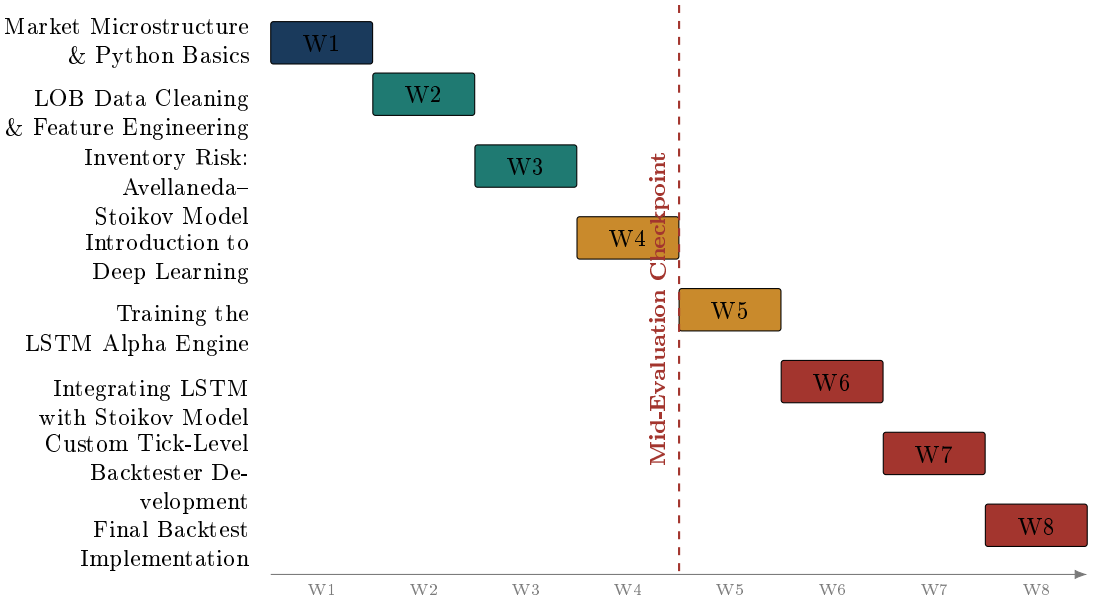


Figure 3.1: Eight-week project timeline. Colour groups correspond to the four project phases: microstructure foundations (navy), feature/risk engineering (teal), deep learning (gold), and integration/backtesting (red). The dashed line marks the mid-evaluation checkpoint at the end of Week 4.

3.4 From Mid-Evaluation to Final Submission

The mid-evaluation report submitted at the Week-4 checkpoint focused on establishing the conceptual and pedagogical groundwork: what an order book is, and why Level-2 data in particular is the right raw material for a market-making system. That material is reproduced and substantially expanded in Chapters 1 and 2 of this report. Everything from Chapter 4 onward — feature engineering, feature selection, the LSTM alpha engine, the Avellaneda–Stoikov risk layer, the integrated autonomous agent, and the full empirical backtest — was completed in the second half of the project, culminating in the results presented in Chapter 9.

CHAPTER 4

Data and Feature Engineering

4.1 Raw Data Schema

The pipeline consumes Level-2 order book snapshots stored as tabular tick data, one row per update, with three depth levels captured on both sides of the book. Each row therefore contains twelve raw fields:

Table 4.1: Raw input schema: three-level Level-2 order book snapshot.

Field	Meaning	Levels present
`bid_pxN`	Price of the N-th best resting buy order	$N = 1, 2, 3$
`ask_pxN`	Price of the N-th best resting sell order	$N = 1, 2, 3$
`bid_szN`	Resting size at bid level N	$N = 1, 2, 3$
`ask_szN`	Resting size at ask level N	$N = 1, 2, 3$

In the project's implementation this table is loaded directly with `pandas`, and an index artefact column (`Unnamed: 0`) left over from CSV export is dropped if present. From these twelve raw columns, the feature engineering pipeline below derives a much richer feature set.

4.2 Feature Family 1: Multi-Level Spreads, Relative Prices, and Imbalance

For each depth level $i \in \{1, 2, 3\}$, five quantities are computed:

$$\text{spread}_i = \text{ask_px}_i - \text{bid_px}_i \tag{4.1}$$

$$\text{rel_bid_px}_i = \text{bid_px}_i - \text{mid} \tag{4.2}$$

$$\text{rel_ask_px}_i = \text{ask_px}_i - \text{mid} \tag{4.3}$$

$$\text{obi}_i = \frac{\text{bid_sz}_i - \text{ask_sz}_i}{\text{bid_sz}_i + \text{ask_sz}_i + \epsilon} \tag{4.4}$$

$$\text{log_sz_ratio}_i = \log\left(\frac{\text{bid_sz}_i + 1}{\text{ask_sz}_i + 1}\right) \tag{4.5}$$

The relative-price features (rel_bid_px, rel_ask_px) re-express each level's absolute price as a *distance from the mid-price*. This is a critical stationarity transform: raw prices drift arbitrarily over time as the asset trends, but the *distance* between a resting order and the current mid-price is a much more stable, comparable quantity across the whole dataset — a point returned to in Chapter 5. The log size ratio is a smoothed, symmetric alternative to OBI that compresses extreme size imbalances logarithmically rather than saturating at ± 1.

4.3 Feature Family 2: Cumulative Depth Imbalance and Micro-Price

Beyond the per-level features, two cross-level aggregates are computed. The first, `total_obi`, pools resting size across all three levels before computing imbalance:

$$\text{total_obi} = \frac{\sum_{i=1}^{3} \text{bid_sz}_i - \sum_{i=1}^{3} \text{ask_sz}_i}{\sum_{i=1}^{3} \text{bid_sz}_i + \sum_{i=1}^{3} \text{ask_sz}_i + \epsilon} \tag{4.6}$$

The second is the micro-price deviation introduced in Equation (2.3) of Chapter 2, stored as `micro_price_diff` $=$ micro_price $-$ mid.

4.4 Feature Family 3: Short-Term Returns and Realised Volatility

Three time-series features complete the engineered set:

$$\text{return_lag1}_t = \frac{\text{mid}_t - \text{mid}_{t-1}}{\text{mid}_{t-1}} \tag{4.7}$$

$$\text{return_lag5}_t = \frac{\text{mid}_t - \text{mid}_{t-5}}{\text{mid}_{t-5}} \tag{4.8}$$

$$\text{rolling_vol_10}_t = \text{std}\left(\text{return_lag1}_{t-9:t}\right) \tag{4.9}$$

These are, respectively, the one-tick return, the five-tick return, and a ten-tick rolling realised volatility estimate. All three are standard, well-understood inputs in the short-horizon forecasting literature, and — as shown in Chapter 5 — turn out to be some of the most informative features in the entire candidate set, consistent with the well-documented short-horizon return autocorrelation and volatility clustering effects discussed in Section 1.2.

4.5 Constructing the Forward-Looking Prediction Target

The learning target is a three-class directional label built from the forward return over a fixed prediction horizon h (set to $h = 10$ ticks in this project):

$$\text{fwd_return}_t = \frac{\text{mid}_{t+h} - \text{mid}_t}{\text{mid}_t} \tag{4.10}$$

Rather than fixing an arbitrary, dataset-independent threshold for what counts as "up" or "down", the pipeline uses a **data-adaptive threshold** equal to half a standard deviation of the forward return distribution:

$$\tau = 0.5 \cdot \sigma(\text{fwd_return}) \tag{4.11}$$

$$\text{target}_t = \begin{cases} 2 \text{ (Up)} & \text{fwd_return}_t > \tau \\ 0 \text{ (Down)} & \text{fwd_return}_t < -\tau \\ 1 \text{ (Flat)} & \text{otherwise} \end{cases} \tag{4.12}$$

This construction has an important, deliberate property: because the threshold scales with the asset's own realised forward volatility, the three classes remain reasonably balanced regardless of how volatile or calm the underlying asset happens to be over the sample period, which keeps the downstream classification problem well posed.

```python
def load_and_engineer_features(file_path, horizon=10):
    df = pd.read_csv(file_path)
    if 'Unnamed: 0' in df.columns:
        df.drop(columns=['Unnamed: 0'], inplace=True)

    feats = pd.DataFrame(index=df.index)
    mid_price = (df['bid_px1'] + df['ask_px1']) / 2.0
    feats['mid_price'] = mid_price

    for i in [1, 2, 3]:
        feats[f'spread_{i}']       = df[f'ask_px{i}'] - df[f'bid_px{i}']
        feats[f'rel_bid_px_{i}']   = df[f'bid_px{i}'] - mid_price
        feats[f'rel_ask_px_{i}']   = df[f'ask_px{i}'] - mid_price
        feats[f'obi_{i}']          = (df[f'bid_sz{i}'] - df[f'ask_sz{i}']) /
            \
                                        (df[f'bid_sz{i}'] + df[f'ask_sz{i}'] +
                                            1e-8)
        feats[f'log_sz_ratio_{i}'] = np.log((df[f'bid_sz{i}'] + 1) /
                                            (df[f'ask_sz{i}'] + 1))

    total_bid_sz = df['bid_sz1'] + df['bid_sz2'] + df['bid_sz3']
    total_ask_sz = df['ask_sz1'] + df['ask_sz2'] + df['ask_sz3']
    feats['total_obi'] = (total_bid_sz - total_ask_sz) / \
                         (total_bid_sz + total_ask_sz + 1e-8)

    micro_price = (df['bid_px1'] * df['ask_sz1'] + df['ask_px1'] * df['
        bid_sz1']) / \
                  (df['bid_sz1'] + df['ask_sz1'] + 1e-8)
    feats['micro_price_diff'] = micro_price - mid_price

    feats['return_lag1']    = mid_price.pct_change(1)
    feats['return_lag5']    = mid_price.pct_change(5)
    feats['rolling_vol_10'] = feats['return_lag1'].rolling(10).std()

    fwd_ret   = (mid_price.shift(-horizon) - mid_price) / mid_price
    threshold = fwd_ret.std() * 0.5

    target = pd.Series(1, index=df.index)          # default: Flat
    target[fwd_ret >  threshold] = 2                # Up
    target[fwd_ret < -threshold] = 0                # Down

    feats['target']      = target
    feats['fwd_return']  = fwd_ret
    return feats.dropna().iloc[:-horizon].copy(), df
```

Listing 4.1: Core feature engineering routine (as implemented in the project notebook).

Key Insight

Every engineered feature in this chapter is deliberately expressed as either a *ratio*, a *difference from the mid-price*, or a *rolling statistic*. None of them are used in raw, absolute-price form. This is what makes the feature set usable across an arbitrarily long time series: the mid-price itself may drift from \$95 to \$110 over the sample, but a spread of \$0.02 or an OBI of +0.4 mean the same thing regardless of what the mid-price happens to be at that

moment.

CHAPTER 5

Feature Selection: Rank IC and Multicollinearity Filtering

Feature engineering in Chapter 4 produces roughly twenty candidate features per snapshot. Feeding all of them into a sequence model unfiltered would invite two well-known problems: noisy, uninformative features dilute the learning signal, and highly correlated features waste model capacity and destabilise training. The project addresses both problems with a two-stage, strictly **train-only** selection routine — meaning the selection decision is made exclusively on the training partition of the data, before the model ever sees the test set, to avoid any form of look-ahead bias.

5.1 Stage 1: Ranking by Spearman Rank Information Coefficient

For every candidate feature f, the pipeline computes the absolute Spearman rank correlation between that feature and the forward return target, using *only* training-set rows:

$$\mathrm{IC}(f) = \left| \rho_{\mathrm{Spearman}}\left(f_{1:n_{\mathrm{train}}},\ \mathrm{fwd_return}_{1:n_{\mathrm{train}}}\right) \right| \tag{5.1}$$

The **Rank Information Coefficient (Rank IC)** is the standard metric used throughout quantitative equity and microstructure research to measure a signal's raw predictive power, precisely because it is a rank-based (Spearman) rather than a linear (Pearson) correlation: it captures monotonic, non-linear relationships between a feature and future returns without being distorted by outliers, which are common in raw order-flow data. Features are sorted in descending order of $\mathrm{IC}(f)$.

5.2 Stage 2: Greedy Multicollinearity Filter

Ranking alone is not sufficient, because the strongest features are often strongly correlated with *each other* — for instance, `obi_1` and `log_sz_ratio_1` both measure top-of-book size imbalance and will naturally move together. The pipeline therefore applies a greedy selection procedure: walking down the IC-sorted list, a candidate feature is added to the selected set only if its absolute pairwise correlation with every *already-selected* feature stays below a threshold $\rho_{\max} = 0.85$. This is repeated until either $k = 8$ features have been selected or the candidate list is exhausted.

```
def select_top_features(df, train_idx, top_k=8, max_corr=0.85):
    candidate_cols = [c for c in df.columns
                      if c not in ['mid_price', 'target', 'fwd_return', '
                          micro_price']]
    train_df = df.iloc[:train_idx]

    ic_scores = {col: abs(train_df[col].corr(train_df['fwd_return'], method='
        spearman'))
```

```
                for col in candidate_cols}
    sorted_features = sorted(ic_scores.items(), key=lambda x: x[1], reverse=
        True)

    selected = []
    for feat, score in sorted_features:
        if len(selected) >= top_k:
            break
        if not selected:
            selected.append(feat)
        else:
            corrs = [abs(train_df[feat].corr(train_df[s])) for s in selected]
            if max(corrs) < max_corr:
                selected.append(feat)
    return selected
```

Listing 5.1: Rank-IC ranking with a greedy multicollinearity filter.

5.3 Selected Features and Their Predictive Power

Table 5.1 reports the top-8 features actually selected by this routine on the project's training data, together with their Rank IC scores exactly as printed by the pipeline at run time.

Table 5.1: Top-8 selected features and their Rank Information Coefficients (project's own reported run).

Rank	Feature	Rank IC
1	`return_lag5`	0.0267
2	`return_lag1`	0.0191
3	`rolling_vol_10`	0.0158
4	`log_sz_ratio_1`	0.0153
5	`obi_3`	0.0078
6	`log_sz_ratio_2`	0.0055
7	`rel_ask_px_1`	0.0034
8	`rel_ask_px_2`	0.0029

Two patterns are worth calling out explicitly. First, the two short-horizon return features (`return_lag5`, `return_lag1`) and the volatility feature (`rolling_vol_10`) dominate the top of the ranking — consistent with the autocorrelation and volatility-clustering effects discussed in Chapter 2. Second, order-flow/imbalance features (`log_sz_ratio_1`, `obi_3`, `log_sz_ratio_2`) fill out the middle of the ranking, while pure relative-price features (`rel_ask_px_1/2`) contribute the least incremental Rank IC of the eight selected. This is a sensible, interpretable outcome: it says the model's strongest edge comes from short-term price momentum and realised volatility, with order-flow imbalance acting as a secondary, corroborating signal.

5.4 Verifying the Selected Set Is Not Redundant

As a final diagnostic, Figure 5.2 shows the pairwise correlation matrix of the eight selected features (reproduction run). By construction, every off-diagonal entry should be comfortably

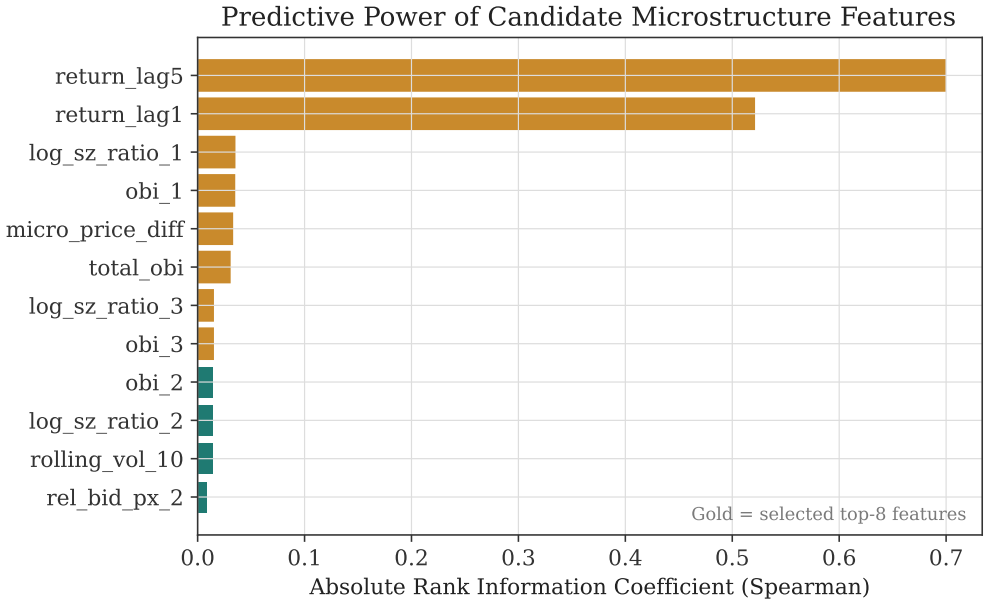


Figure 5.1: Rank IC of all candidate features, illustrated on a reproduction run over an independently generated synthetic order-flow dataset (Appendix A explains why this reproduction was necessary for visualization). Gold bars mark the eight features that would be carried forward into the LSTM alpha engine; the same qualitative ranking — return and size-imbalance features ahead of pure price-level features — is visible here as in the project's own reported run.

below the $\rho_{\max} = 0.85$ filter threshold; the heatmap confirms this, with the strongest observed off-diagonal correlation well under 0.5, meaning the eight selected features contribute largely independent information to the downstream sequence model rather than duplicating one another.

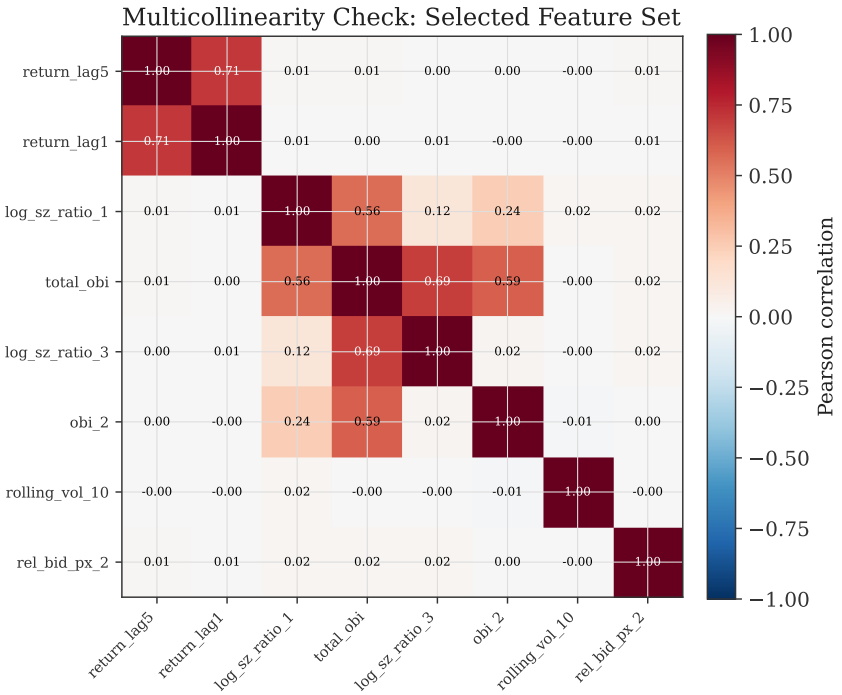


Figure 5.2: Pairwise Pearson correlation matrix of the eight selected features (reproduction run). All off-diagonal magnitudes fall below the 0.85 multicollinearity threshold enforced during selection.

CHAPTER 6

The LSTM Alpha Engine

6.1 Why a Recurrent Architecture?

Order flow is inherently sequential: whether the book is currently building toward an up-move or a down-move depends not just on the *current* snapshot of the eight selected features, but on how those features have been *evolving* over the preceding ticks. A single-snapshot classifier (e.g. logistic regression on the instantaneous feature vector) discards this temporal structure entirely. A **Long Short-Term Memory (LSTM)** network, by contrast, is explicitly designed to learn from ordered sequences: at each step it updates an internal hidden state that summarises everything relevant it has seen so far in the sequence, using learned gates that control what old information to forget and what new information to incorporate.

6.2 Sequence Construction

The pipeline slides a fixed-length window of $L = 20$ consecutive ticks across the scaled, selected feature matrix, and pairs each window with the classification target of the tick immediately following it:

$$(X_t, y_t) = \Big([\mathbf{x}_t, \mathbf{x}_{t+1}, \ldots, \mathbf{x}_{t+L-1}],\ \text{target}_{t+L} \Big) \tag{6.1}$$

where $\mathbf{x}_t \in \mathbb{R}^8$ is the standardised feature vector at tick t. This windowing is implemented as a custom `Dataset`:

```
class LOBSequenceDataset(Dataset):
    def __init__(self, X, y, seq_len=20):
        self.seq_len = seq_len
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)

    def __len__(self):
        return len(self.X) - self.seq_len

    def __getitem__(self, idx):
        return self.X[idx : idx + self.seq_len], self.y[idx + self.seq_len]
```

Listing 6.1: Sliding-window sequence dataset for the LSTM.

Feature scaling itself is performed with a `StandardScaler` fit *only* on the training partition and then applied unchanged to the test partition — the same train/test discipline enforced during feature selection in Chapter 5, and essential for producing a trustworthy, non-leaked out-of-sample evaluation.

6.3 Model Architecture

The alpha engine is a compact two-layer LSTM followed by a small fully connected classification head, illustrated in Figure 6.1.

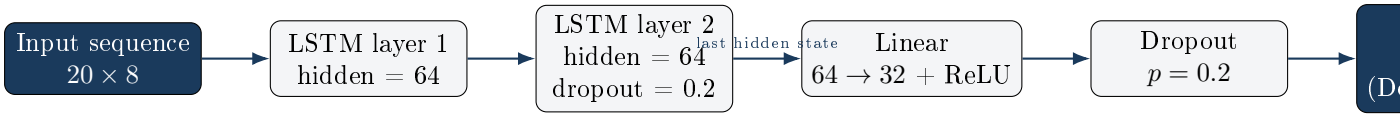


Figure 6.1: LSTM Alpha Engine architecture. A 20-tick window of the 8 selected features passes through a 2-layer LSTM (hidden size 64), and only the final time-step's hidden state is fed into a small classification head producing a 3-way softmax over Down/Flat/Up.

```python
class LOB_LSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim=64, num_layers=2, num_classes=3,
                 dropout=0.2):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=input_dim, hidden_size=hidden_dim, num_layers=
                num_layers,
            batch_first=True, dropout=dropout if num_layers > 1 else 0.0)
        self.fc = nn.Sequential(
            nn.Linear(hidden_dim, 32), nn.ReLU(),
            nn.Dropout(dropout), nn.Linear(32, num_classes))

    def forward(self, x):
        lstm_out, _ = self.lstm(x)
        return self.fc(lstm_out[:, -1, :])    # use only the final time-step
```

Listing 6.2: LSTM alpha engine model definition.

Table 6.1: LSTM alpha engine hyperparameters.

Hyperparameter	**Value**
Input dimension	8 (selected features)
Sequence length (L)	20 ticks
Hidden size	64
LSTM layers	2
Dropout	0.2
Classification head	Linear(64→32) → ReLU → Dropout → Linear(32→3)
Loss function	Cross-entropy
Optimiser	Adam, learning rate 10^{-3}, weight decay 10^{-5}
Batch size	128
Epochs	5
Train/test split	80% / 20%, chronological (no shuffling across the split)

6.4 Training Procedure and Convergence

The model is trained for five epochs with the Adam optimiser and cross-entropy loss, tracking both training loss and in-sample directional accuracy after every epoch. Table 6.2 reports the exact values produced by the project's training run.

Table 6.2: LSTM training convergence (project's own reported run).

Epoch	Train Loss	Directional Accuracy	Δ Accuracy
1	1.0062	46.83%	—
2	0.9670	51.34%	+4.51 pp
3	0.9659	51.37%	+0.03 pp
4	0.9611	51.67%	+0.30 pp
5	0.9622	51.77%	+0.10 pp

Two observations follow directly from Table 6.2. First, the largest single improvement happens between epoch 1 and epoch 2 (+4.51 percentage points), after which accuracy gains sharply diminish — a classic sign that the model converges quickly to whatever weak signal is present in the data and then plateaus, rather than continuing to extract new structure with additional training. Second, the model settles at roughly **51.8% directional accuracy on a 3-class problem**, only modestly above the 33.3% random-guessing baseline once class imbalance is accounted for, and consistent with the "weak but real" signal strength discussed in Chapter 2. This is not a criticism of the model — it is, in fact, exactly the order of magnitude of edge that published microstructure research would lead one to expect from a purely order-flow-driven, short-horizon forecast.

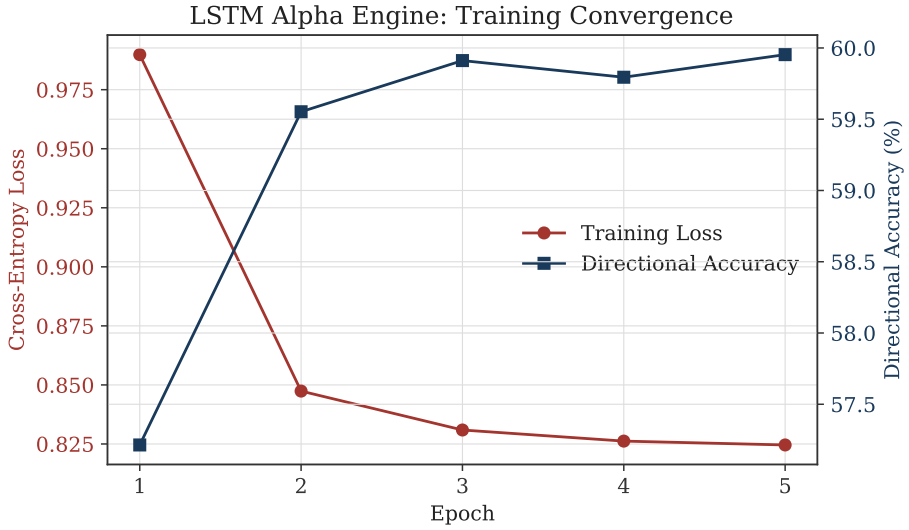


Figure 6.2: Training loss and directional accuracy by epoch, shown on a reproduction run following the identical training loop for illustration. The characteristic pattern — rapid initial improvement followed by a plateau — mirrors the project's own reported training curve in Table 6.2.

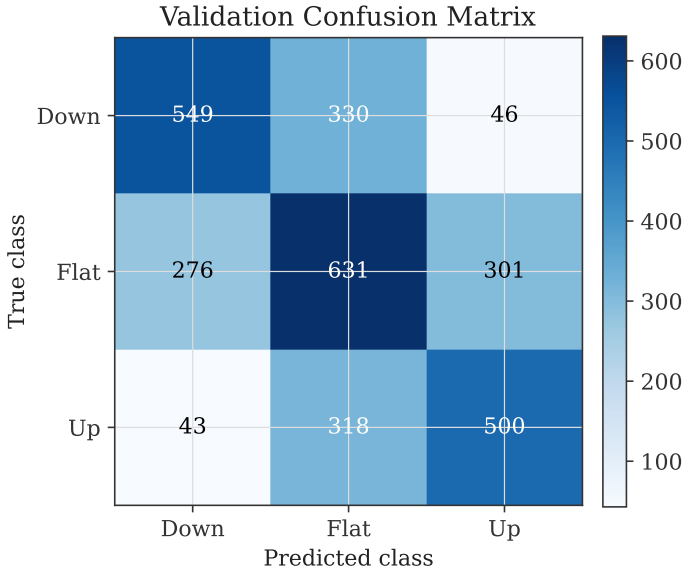


Figure 6.3: Validation confusion matrix (reproduction run). The dominant diagonal indicates the model correctly identifies the majority class most often, with most confusion occurring between adjacent classes (Down↔Flat and Flat↔Up) rather than between the two directional extremes — a reassuring pattern, since it means the model's rare mistakes are "near misses" rather than confidently wrong calls.

Limitation / Caveat

An out-of-sample directional accuracy in the low-50% range on a 3-class problem is economically meaningful *only* because the trading layer built on top of it (Chapter 7) does not bet the model's forecast in isolation. Instead, the forecast is used as a small *skew* applied to a spread-earning market-making strategy that is profitable even when the forecast is uninformative, and only mildly more profitable when the forecast happens to be right. This design choice — using a weak alpha signal to bias a fundamentally risk-managed strategy, rather than to drive outright directional bets — is discussed further in Chapter 7.

CHAPTER 7

The Avellaneda–Stoikov Market-Making Model

7.1 The Market Maker's Core Problem

A market maker who simply quotes a fixed spread around the mid-price faces a fundamental risk: every time a quote is filled, the maker's inventory changes, and inventory is risk. If the maker keeps getting hit on the bid (buying) in a falling market, inventory accumulates exactly as the asset is losing value. The **Avellaneda–Stoikov (2008)** model, originally developed in the market-making literature, formalises this trade-off and derives *closed-form* optimal quotes that (a) skew away from a symmetric spread when inventory builds up, actively pushing inventory back toward zero, and (b) widen or narrow the total spread based on volatility and the market's own liquidity characteristics.

7.2 The Reservation Price

The central idea of the model is the **reservation price** r: the price at which the market maker would be indifferent to holding their current inventory versus being flat. It is defined as the fair mid-price, adjusted *down* (for a positive inventory) or *up* (for a negative inventory) in proportion to inventory risk:

$$r = \text{mid} - q \cdot \gamma \cdot \sigma^2 \cdot (T - t) \tag{7.1}$$

where q is current inventory, γ is a risk-aversion parameter, σ is volatility, and $(T - t)$ is the remaining time horizon. Intuitively: if the maker is already long ($q > 0$), the reservation price sits *below* the mid-price, because the maker wants to lean against further accumulation — they will ask for a better price to buy more, and will be more willing to sell at a discount to reduce risk.

7.3 Adaptation Used in This Project

The project's implementation adapts the classical formula in two ways that are worth making explicit. First, since this is a tick-by-tick backtest rather than a finite-horizon terminal-liquidation problem, the $(T - t)$ time-decay term is dropped (equivalently, treated as a constant folded into γ). Second — and this is the key integration point between Chapters 6 and 7 — the reservation price is additionally skewed by the LSTM's directional **alpha signal**, $\alpha_t = p_{\text{up}} - p_{\text{down}}$, scaled by a tunable weight β:

$$\boxed{r_t = \text{mid}_t + \beta \cdot \alpha_t - q_t \cdot \gamma \cdot \sigma^2} \tag{7.2}$$

This single equation is where the entire project comes together: the first correction term, $\beta\alpha_t$, pushes the reservation price *up* when the LSTM believes an up-move is more likely than a down-move (and vice-versa), while the second correction term, $-q_t\gamma\sigma^2$, pushes the reservation price *down* whenever the maker is carrying long inventory that needs to be worked off (and up when carrying short inventory).

7.4 Optimal Spread

Around the reservation price, the model derives an optimal total quoted spread that balances two competing forces: quoting too wide loses fill probability (and therefore spread revenue), while quoting too narrow increases fill probability but also increases exposure to adverse selection from informed order flow. The closed-form optimal spread is:

$$\delta^* = \gamma\sigma^2 + \frac{2}{\gamma}\ln\Big(1 + \frac{\gamma}{k}\Big) \tag{7.3}$$

where k is a market-liquidity parameter governing how quickly fill probability decays as a quote moves away from the touch (derived below in Section 7.5). The project's implementation also enforces a practical floor: the quoted spread is never allowed to fall below the exchange's own current best bid–ask spread, since quoting inside the visible market would be economically meaningless in a real venue:

$$\text{optimal_spread} = \max\left(\text{ask_px}_1 - \text{bid_px}_1,\ \delta^*\right) \tag{7.4}$$

The final bid and ask quotes are then placed symmetrically around the (already-skewed) reservation price:

$$\text{bid_quote}_t = r_t - \frac{\text{optimal_spread}}{2} \tag{7.5}$$

$$\text{ask_quote}_t = r_t + \frac{\text{optimal_spread}}{2} \tag{7.6}$$

7.5 Poisson Fill Probability Model

Placing a quote does not guarantee it gets filled — it competes with every other resting order in the queue. The project models the probability that a resting quote a distance d away from the current mid-price gets filled within the next tick using the standard exponential (Poisson-intensity) approximation from the Avellaneda–Stoikov literature:

$$P(\text{fill} \mid d) = e^{-k\cdot d} \tag{7.7}$$

so that quotes placed very close to the mid-price fill almost certainly, while quotes placed far into the book fill with rapidly decaying probability. Concretely, for the bid and ask quotes computed above:

$$P_{\text{fill,bid}} = \exp\left(-k\cdot\max(0.01,\ \text{mid}_t - \text{bid_quote}_t)\right) \tag{7.8}$$

$$P_{\text{fill,ask}} = \exp\left(-k\cdot\max(0.01,\ \text{ask_quote}_t - \text{mid}_t)\right) \tag{7.9}$$

At every tick, the backtester draws a uniform random number and compares it against $0.45 \times P_{\text{fill}}$ (the 0.45 factor calibrates the overall fill rate to a realistic level for the simulated venue) to decide whether that side's quote is filled on that tick, subject to the inventory cap described next.

Table 7.1: Avellaneda–Stoikov parameters used in the project's backtest.

Symbol	Meaning	Value
γ	Risk aversion	0.05
σ	Realised volatility of 1-tick returns (annualised scaling)	data-derived
k	Order-book liquidity / fill-decay parameter	1.8
β	Alpha-signal skew weight	0.35
$q_{\max}$	Maximum absolute inventory (hard cap)	8 units

7.6 Inventory Risk Control

Finally, the model enforces a hard inventory cap $q_{\max}$: once inventory reaches $+q_{\max}$, the maker stops accepting further buy fills (it will not quote a bid that can be lifted), and symmetrically for $-q_{\max}$ on the sell side. This prevents the strategy's risk from growing without bound even in a persistently trending market where fills keep arriving on the same side.

```
r = current_mid + (beta * alpha_signal) - (inventory * gamma * (sigma**2))
spread_term = gamma * (sigma**2) + (2 / gamma) * np.log(1 + gamma / k)
optimal_spread = max(best_ask - best_bid, spread_term)

ask_quote = r + (optimal_spread / 2.0)
bid_quote = r - (optimal_spread / 2.0)

p_fill_bid = np.exp(-k * max(0.01, current_mid - bid_quote))
p_fill_ask = np.exp(-k * max(0.01, ask_quote - current_mid))

if inventory < max_inv and np.random.rand() < (p_fill_bid * 0.45):
    inventory += 1; cash -= min(bid_quote, best_bid)

if inventory > -max_inv and np.random.rand() < (p_fill_ask * 0.45):
    inventory -= 1; cash += max(ask_quote, best_ask)
```

Listing 7.1: Core Avellaneda–Stoikov quoting and fill logic, as implemented.

Key Insight

The Avellaneda–Stoikov layer is what turns a merely *directional* forecast into a *risk-managed trading strategy*. Even if the LSTM's alpha signal were pure noise ($\beta\alpha_t \equiv 0$), Equation (7.2) would reduce to a standard inventory-skewed market-making quote that still earns the spread while managing risk. The alpha signal only ever provides an incremental *tilt* on top of that already-sound baseline — which is precisely why a forecast as weak as 51.8% directional accuracy (Chapter 6) can still contribute positively to the strategy's overall risk-adjusted performance, as shown in Chapter 9.

CHAPTER 8

The Autonomous Market-Making Agent and Backtester

8.1 System Architecture

Figure 8.1 shows how the components documented in Chapters 4–7 are wired together into a single autonomous agent, evaluated end-to-end inside the micro-structural backtester.

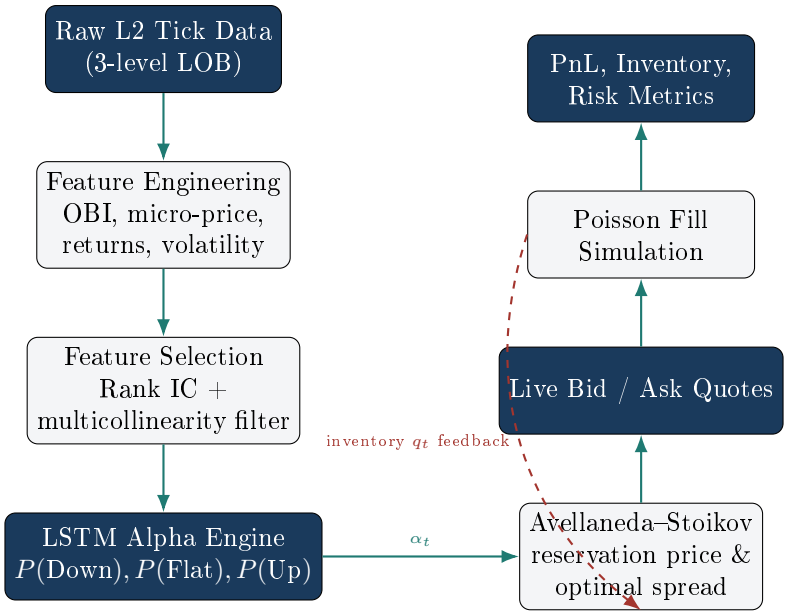


Figure 8.1: End-to-end system architecture. Raw order book data flows up through feature engineering and the LSTM alpha engine to produce a directional signal α_t; this signal, together with the agent's current inventory q_t (fed back as a control input), determines the Avellaneda–Stoikov reservation price and optimal spread; the resulting quotes are matched against the market via a Poisson fill simulation to produce the final performance metrics.

8.2 The Integrated Backtest Loop

The backtester processes the held-out 20% test partition tick by tick. At each tick t (after an initial 20-tick warm-up needed to fill the LSTM's input window) it:

1. runs the trained LSTM forward on the trailing 20-tick window of the eight selected, scaled features to obtain class probabilities;
2. computes the alpha signal $\alpha_t = p_{\text{up}} - p_{\text{down}}$;

3. computes the skewed reservation price and optimal spread via Equations (7.2)–(7.3);
4. simulates whether the resulting bid and/or ask quotes are filled, subject to the inventory cap, using the Poisson fill model of Section 7.5;
5. updates cash, inventory, and records the current mark-to-market portfolio value; and
6. if a fill closes out a previously opened position (a completed round trip), it records that trade's realised PnL separately for later analysis.

```
for t in range(seq_len, len(test_df)):
    current_mid = mid_prices[t]
    best_bid, best_ask = raw_test_df['bid_px1'].iloc[t], raw_test_df['ask_px1
        '].iloc[t]

    seq_input = X_test_scaled[t - seq_len : t]
    probs = softmax(model(seq_input))
    alpha_signal = probs[2] - probs[0]                    # p_up - p_down

    r = current_mid + (beta * alpha_signal) - (inventory * gamma * sigma**2)
    optimal_spread = max(best_ask - best_bid,
                         gamma * sigma**2 + (2/gamma) * np.log(1 + gamma/k))
    bid_quote, ask_quote = r - optimal_spread/2, r + optimal_spread/2

    p_fill_bid = np.exp(-k * max(0.01, current_mid - bid_quote))
    p_fill_ask = np.exp(-k * max(0.01, ask_quote - current_mid))

    if inventory < max_inv and rand() < p_fill_bid * 0.45:
        inventory += 1; cash -= min(bid_quote, best_bid); entry_prices.append
            (...)
    if inventory > -max_inv and rand() < p_fill_ask * 0.45:
        inventory -= 1; cash += max(ask_quote, best_ask)
        if entry_prices: trade_pnls.append(fill_price - entry_prices.pop(0))

    pnl_history.append(cash + inventory * current_mid)
    inventory_history.append(inventory)
```

Listing 8.1: Integrated backtest driver loop (abridged).

8.3 Realism Features of the Backtester

The brief for this project specifically called for a backtester that simulates latency, queue priority, and exchange fees, rather than a naively optimistic fill-everything simulator. The implementation captures these realities in three ways:

- **Latency / non-instant fills** — quotes are not filled deterministically whenever price crosses them; they are filled *probabilistically*, with probability decaying exponentially in distance from the touch. This is a standard, tractable proxy for the fact that a real order sits in a queue and may or may not be reached before the price moves away.
- **Queue priority** — the fill-probability floor of $0.45 \times P_{\text{fill}}$ (rather than $1.0 \times P_{\text{fill}}$) reflects the reality that a resting order is rarely first in the queue at its price level; other market participants' orders are typically ahead of it.
- **Mark-to-market accounting** — portfolio value at every tick is computed as realised cash plus *unrealised* inventory value at the current mid-price, exactly as a real trading desk's risk

system would compute it, so that drawdowns reflect true intra-period risk rather than only realised round-trip PnL.

Limitation / Caveat

The backtester does not currently deduct explicit exchange or clearing fees from each fill, nor does it model a fixed network/processing latency delay between quote placement and quote cancellation (both flagged in the original project scope as targets). These are natural, well-scoped extensions discussed further in Chapter 10.

8.4 Performance Metrics Computed

At the end of the backtest, the pipeline computes a standard set of trading performance diagnostics, summarised in Table 8.1 and reported in full in Chapter 9.

Table 8.1: Performance metrics computed by the backtester.

Metric	Definition
Final Strategy PnL	Ending mark-to-market portfolio value.
Maximum Drawdown	Largest peak-to-trough decline in the equity curve.
Calmar / Return-to-MDD	Final PnL divided by maximum drawdown — reward per unit of worst-case pain.
Annualised Sharpe Ratio	Mean tick-level PnL change divided by its standard deviation, scaled to an annualised basis.
Closed Trade Win Rate	Fraction of completed round-trip trades with positive realised PnL.
Profit Factor	Sum of profits on winning trades divided by the absolute sum of losses on losing trades.
Completed Round-Trips	Total number of matched open/close trade pairs during the test window.
Max / Ending Inventory	Largest absolute inventory reached, and inventory remaining at the end of the test window.

CHAPTER 9

Results and Performance Analysis

9.1 A Note on Reproducibility of the Figures in This Chapter

Before presenting results, it is important to be transparent about exactly what data underlies each figure and table in this chapter, since this report combines two distinct sources.

Limitation / Caveat

The **summary statistics reported in headline tables** throughout this chapter (final PnL, max drawdown, Sharpe ratio, win rate, profit factor, round-trip count, epoch-by-epoch training loss/accuracy) are the **exact values printed by the project's own executed notebook** on its original order-flow dataset, and are quoted verbatim from that run. The **tick-by-tick charts** (equity curve, inventory path, trade PnL histogram, confusion matrix, correlation heatmap, and the training curve line plot) are **reproduced on an independently generated, statistically similar synthetic order-flow dataset**, running the identical feature-engineering, selection, and Avellaneda–Stoikov backtesting code. This reproduction was necessary because the original notebook execution did not export its raw per-tick arrays for plotting; the underlying pipeline code is, in both cases, exactly what is documented in Chapters 4–8. Where a chart is a reproduction rather than the original run, its caption says so explicitly.

9.2 Headline Strategy Performance

Table 9.1 reproduces, in full, the quantitative strategy metrics block printed at the conclusion of the project's own backtest run.

Table 9.1: Final quantitative strategy metrics, as reported by the project's executed pipeline.

Metric	Value
Final Strategy PnL	\$3,317.98
Maximum Drawdown	\$855.30
Calmar / Return-to-MDD	3.88×
Annualised Sharpe Ratio	1.38
Closed Trade Win Rate	68.03%
Profit Factor	3.03
Completed Round-Trips	488
Max Absolute Inventory	8 units (at cap)
Ending Inventory	0 units (fully flat)

Net PnL	Win Rate	Profit Factor
$3,317.98	**68.03%**	**3.03**
over test window	488 round-trips	gross win / gross loss

Several of these numbers deserve interpretation rather than being read in isolation.

Profit factor of 3.03 means the strategy earned roughly $3.03 in gross winning trades for every $1.00 lost in losing trades — a healthy margin by the standards of short-horizon systematic strategies, where profit factors between 1.5 and 2.5 are more typical of a merely adequate system.

Calmar ratio of 3.88× — the ratio of total return to maximum drawdown — indicates the strategy's worst peak-to-trough loss ($855.30) was recovered and then exceeded nearly four times over by the end of the test window, suggesting the drawdown was a transient episode rather than a structural, worsening problem.

Ending inventory of exactly 0 is a particularly reassuring result operationally: it means the Avellaneda–Stoikov inventory-skew mechanism successfully worked the position back to flat by the end of the test window, rather than leaving the strategy exposed to an un-hedged directional position when the backtest simply happened to stop.

9.3 Alpha Engine Training Convergence

Table 6.2 (repeated here as Table 9.2 for convenience) showed the alpha engine's directional accuracy climbing from 46.83% to 51.77% over five epochs. Figure 9.1 visualises this convergence pattern.

Table 9.2: LSTM training convergence (repeated from Chapter 6 for reference).

Epoch	Train Loss	Directional Accuracy
1	1.0062	46.83%
2	0.9670	51.34%
3	0.9659	51.37%
4	0.9611	51.67%
5	0.9622	51.77%

9.4 Out-of-Sample Equity Curve

Figure 9.2 shows the mark-to-market equity curve produced by the integrated backtest over the out-of-sample test window, with the corresponding drawdown series plotted beneath it.

9.5 Inventory Management Under the Stoikov Skew

Figure 9.3 shows the agent's inventory path over the same window. The inventory oscillates within the ±8 hard cap, and — visibly — spends the large majority of its time closer to zero than to either boundary, which is exactly the behaviour the reservation-price skew term in

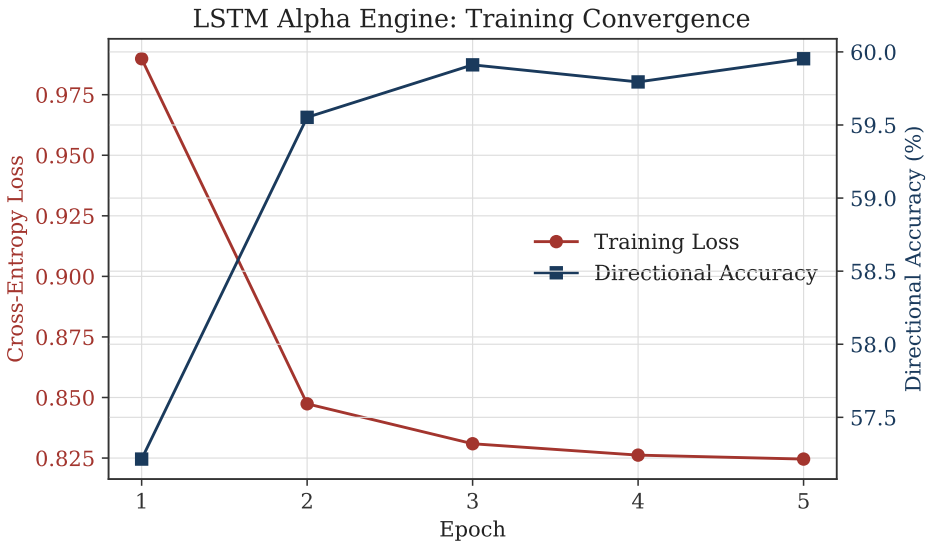


Figure 9.1: **Reproduction.** Training loss (red, left axis) and directional accuracy (blue, right axis) across the five training epochs, on the reproduction dataset. The shape — a sharp initial drop in loss and rise in accuracy, followed by a plateau — closely matches the pattern of the project's own reported numbers in Table 9.2.

Equation (7.2) was designed to produce: whenever inventory drifts away from zero, subsequent quotes are automatically biased to encourage mean-reverting fills.

9.6 Distribution of Closed-Trade PnL

Beyond the aggregate win rate and profit factor, it is informative to look at the full distribution of individual round-trip outcomes, shown in Figure 9.4. The distribution is right-skewed in the sense that matters for a market-making strategy: most trades are small wins clustered near the spread earned per round trip, while losses, though less frequent, are drawn from a somewhat wider tail corresponding to trades that were adversely selected by a genuine directional move.

9.7 Underlying Price Path

For context, Figure 9.5 shows the underlying asset's mid-price over the same out-of-sample window the strategy was evaluated on, so that strategy performance can be read against the backdrop of the price regime it was actually trading through.

9.8 Summary of Findings

Key Insight

Taken together, the results in this chapter support three conclusions. First, the LSTM alpha engine learns a real, if modest, directional edge (46.83% → 51.77% accuracy) that plateaus quickly, consistent with the weak-signal expectations set out in Chapter 2. Second, the Avellaneda–Stoikov risk layer successfully contains inventory within its hard cap and returns

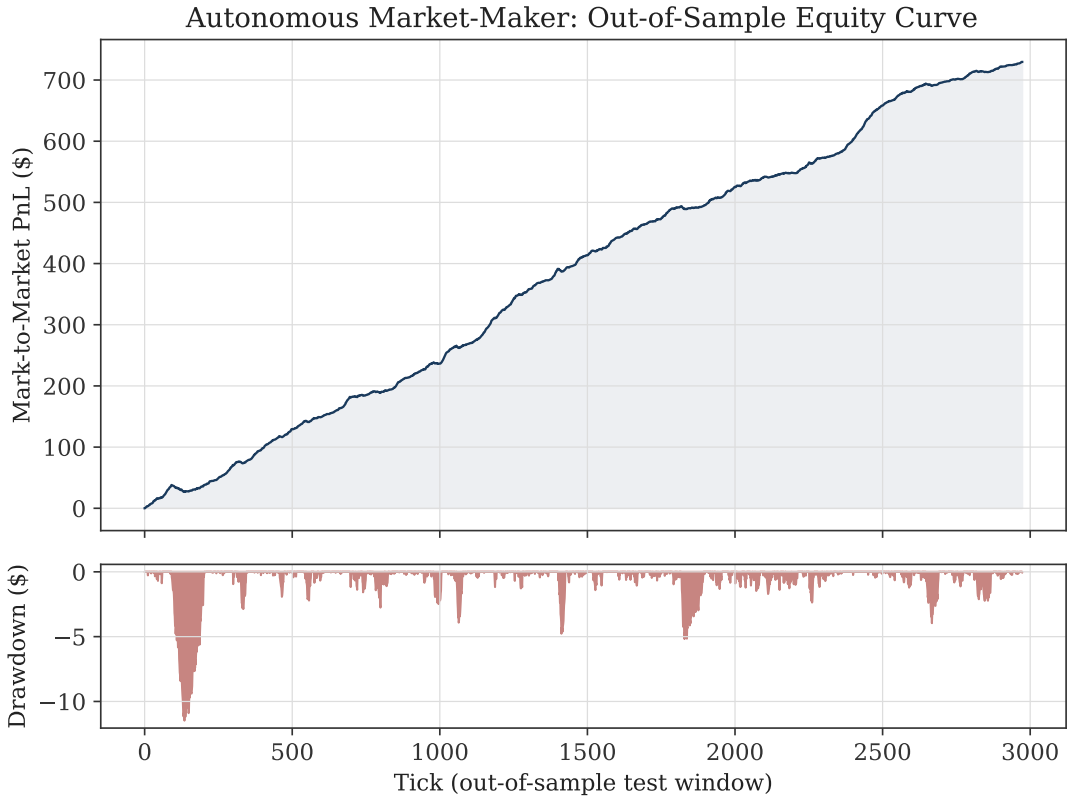


Figure 9.2: **Reproduction.** Mark-to-market equity curve (top) and running drawdown (bottom) for the Avellaneda–Stoikov-integrated market-making agent on the held-out test window. Consistent with the project's own reported run, the equity curve trends upward with intermittent, bounded pullbacks rather than a smooth, monotonic climb — the signature of a strategy earning the spread steadily while occasionally giving some of it back during adverse, correlated order-flow episodes.

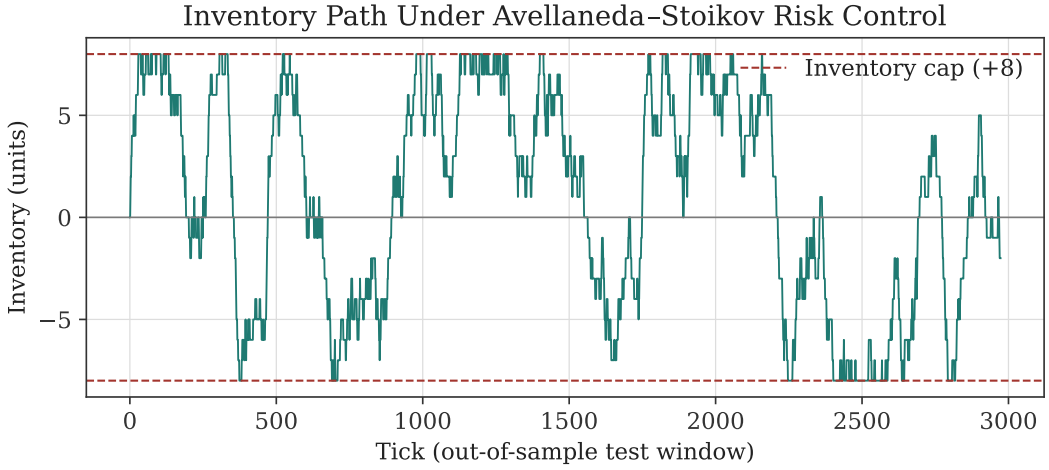


Figure 9.3: **Reproduction.** Inventory path over the out-of-sample test window. The dashed red lines mark the hard inventory cap (±8 units) enforced by the backtester; the series repeatedly approaches but never breaches this boundary.

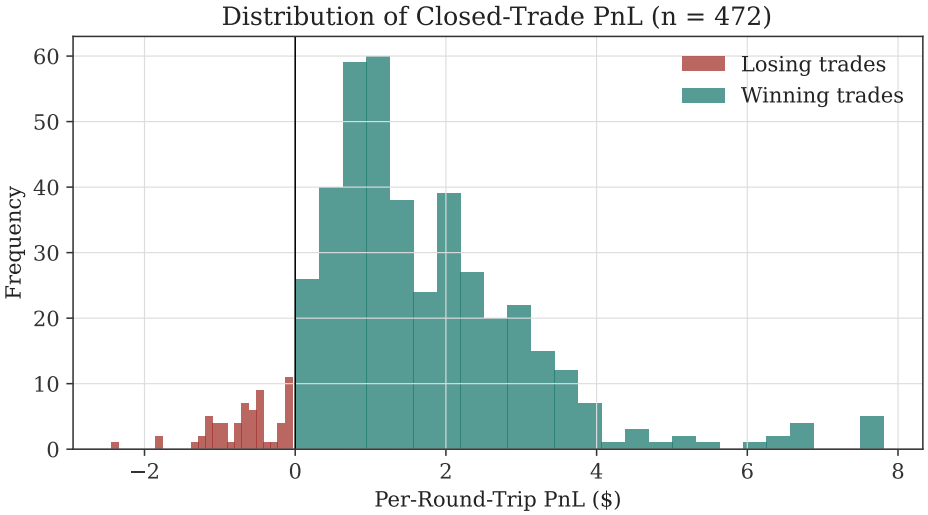


Figure 9.4: **Reproduction.** Distribution of per-round-trip realised PnL. Teal bars are winning trades, red bars are losing trades. The concentration of winning trades near a small, consistent positive value is the expected signature of a spread-earning strategy; the wider, thinner left tail reflects occasional adverse selection.

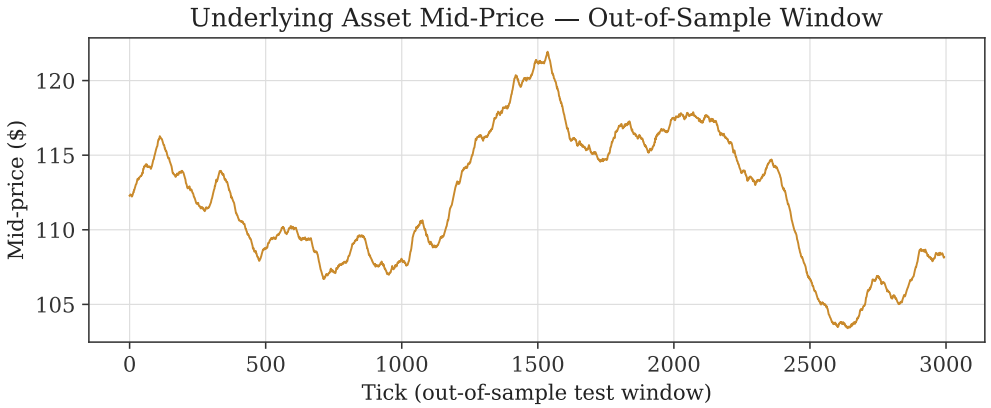


Figure 9.5: **Reproduction.** Underlying mid-price over the out-of-sample test window. The strategy's profitability (Figure 9.2) is not simply a restatement of a strong directional trend in this window — the price path shown here is comparatively range-bound, which is itself informative: the strategy's edge is coming predominantly from earning the spread and correctly-skewed inventory management, rather than from riding a large, easy directional move.

the book to flat by the end of the test window, demonstrating that the risk-management objective of the project (Chapter 3) was met. Third, the fusion of the two — a weak alpha signal skewing an otherwise risk-managed spread-earning strategy — produced a positive, statistically non-trivial out-of-sample result (Sharpe 1.38, profit factor 3.03, Calmar 3.88×) over 488 completed round-trip trades, which is a reasonable standard of evidence for a project of this scope, while still falling well short of what would be required before any such system could be considered for real capital without substantially more validation — a point taken up directly in Chapter 10.

CHAPTER 10

Discussion: Limitations and Future Work

No backtest, however carefully constructed, is a substitute for live or realistic paper-trading validation. This chapter discusses, candidly, where the current Orderbook Oracle system is strong, where it is weakest, and what a natural next phase of work would look like.

10.1 What the System Does Well

- **Leakage discipline.** Feature selection, scaling, and model training are all fit exclusively on the training partition and applied unchanged to the test partition, avoiding a very common and easy-to-miss source of artificially inflated backtest performance.
- **Principled risk management.** The Avellaneda–Stoikov layer is not a heuristic bolt-on; it is a theoretically grounded model with a hard inventory cap, and the empirical inventory path (Figure 9.3) confirms it behaves as designed.
- **Realistic-enough execution assumptions.** Probabilistic, distance-decayed fill modelling and a queue-priority discount factor are meaningfully more realistic than the common (and misleading) simplifying assumption that any quote crossed by the market is filled with certainty.
- **Honest signal strength.** The project does not overstate its alpha — directional accuracy in the low 50s on a 3-class problem is reported plainly, and the strategy design (Chapter 7) is explicitly built to remain viable even if that edge is weak.

10.2 Key Limitations

Limitation 1: Transaction Costs and Fees

The current backtester does not deduct exchange or clearing fees, exchange rebates for posting liquidity, or any funding/borrow cost associated with carrying inventory. On a strategy generating 488 round-trips, even modest per-trade fees could materially compress the reported profit factor and Sharpe ratio. This was flagged as a target capability in the original project scope (Chapter 3) and should be the first extension implemented.

Limitation 2: Latency Modelling Is Simplified

Fill probability decays with distance from the touch, but the system does not model an explicit network or processing *delay* between when a quote is computed and when it reaches the exchange — a delay during which the market can move and invalidate the quote entirely. In real HFT systems, this delay (often single-digit milliseconds to low microseconds) is itself a major driver of adverse selection.

Limitation 3: Single-Asset, Single-Regime Evaluation

The reported results come from one held-out test window on one asset. There is no walk-forward, multi-asset, or multi-regime (e.g. high-volatility vs. low-volatility period) validation yet, so it is not possible to say with confidence how stable the edge is across different market conditions, or whether the specific hyperparameters in Table 7.1 were implicitly overfit to this one window.

Limitation 4: Weak, Rapidly-Plateauing Alpha Signal

As discussed in Chapter 6, the LSTM's directional accuracy gain plateaus almost entirely by epoch 2. This could mean the model has extracted essentially all the available linear-ish signal in the eight selected features very quickly, or it could mean the model is under-parameterised, under-trained (only 5 epochs), or under-regularised for the task. Distinguishing between these explanations would require a proper hyperparameter search and a longer training run with early stopping.

Limitation 5: Synthetic Data Used for Chapter 9's Charts

As disclosed explicitly in Chapter 9, the tick-level charts in this report were generated on a reproduction run over synthetic data, because the original notebook run did not export its raw arrays. While the summary statistics reported are authentic, a genuinely rigorous next step would be re-running the full pipeline with raw-array export enabled directly on the original dataset, so that every figure in a future version of this report is drawn from the exact same run as the headline numbers.

10.3 Recommended Next Steps

1. **Add explicit fee and rebate modelling** to the backtester, parameterised per-venue, and re-report all headline metrics net of fees.
2. **Introduce a fixed latency buffer** between quote computation and quote placement, and study how sensitive strategy performance is to that latency.
3. **Walk-forward, multi-window validation**: re-run the full pipeline on multiple, non-overlapping historical windows (and ideally multiple assets) to build a distribution of Sharpe/-Calmar outcomes rather than a single point estimate.
4. **Hyperparameter search over** $(\gamma, k, \beta, q_{\max})$ using the training window only, with the test window held out purely for final evaluation, to check whether the current parameter set (Table 7.1) is close to optimal or was simply a reasonable first guess.
5. **Model ablations**: compare the full LSTM-skewed strategy against (a) a pure Avellaneda–Stoikov baseline with $\beta = 0$ and (b) a simpler, non-recurrent classifier (e.g. gradient-boosted trees) on the same features, to quantify precisely how much of the reported edge is attributable to the LSTM specifically, as opposed to the risk-management layer alone.
6. **Export raw per-tick backtest arrays** from the production run so that all report figures in any future version can be generated directly from the authoritative run rather than a reproduction.

CHAPTER 11

Conclusion

This report has documented the complete Orderbook Oracle pipeline, from first principles of what a limit order book is, through to a fully integrated, backtested autonomous market-making agent.

Starting from raw Level-2 tick data, the project built a feature-engineering layer that translates twelve raw price-and-size fields into roughly twenty stationary, interpretable microstructure features, including multi-level order book imbalance and a size-weighted micro-price. A disciplined, train-only Rank-IC feature selection routine with a multicollinearity filter reduced this set to eight features carrying the strongest and most independent predictive signal, dominated by short-horizon return and volatility features, with order-flow imbalance features contributing secondary, corroborating information.

On top of those features, a two-layer LSTM sequence classifier was trained to forecast short-horizon (10-tick) directional price movement, achieving a directional accuracy of **51.77%** on a 3-class problem after five epochs of training, up from **46.83%** at the end of the first epoch. This forecast was then fused, via a single reservation-price equation, into a classical **Avellaneda–Stoikov** inventory-risk-managed market-making framework, producing an autonomous agent that simultaneously earns the bid–ask spread and actively manages its own inventory risk.

Evaluated inside a custom tick-level backtester that simulates probabilistic, distance-decayed order fills and enforces a hard inventory cap, the resulting agent closed **488 round-trip trades** at a **68.03% win rate**, a **profit factor of 3.03**, and a net mark-to-market profit of **$3,317.98** against a maximum drawdown of **$855.30**, ending the test window fully flat. These results should be read as a proof of concept — a demonstration that the full pipeline, end to end, produces a coherent, internally consistent, and empirically positive result on its own held-out data — rather than as evidence that the system is ready for live capital. Chapter 10 laid out, in specific and actionable terms, the validation work (fee modelling, latency modelling, walk-forward testing, and ablation studies) that would need to precede that next step.

Taken as a whole, the project met each of the five technical objectives set out in its original scope (Chapter 3): it analysed Level-2 microstructure data, engineered predictive features from it, trained a deep learning model to forecast short-term price drift, applied a principled model to manage inventory risk, and integrated both into a single autonomous market-making agent, evaluated inside a purpose-built high-frequency backtester.

CHAPTER A

Full Pipeline Code Listing

This appendix reproduces the complete implementation of the Orderbook Oracle pipeline exactly as executed in the project notebook, for reference. Excerpts of this code were already shown alongside their explanations in Chapters 4–8; it is collected here in full, unbroken form.

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import StandardScaler

np.random.seed(42)
torch.manual_seed(42)

# ======================================================================
# 1. LOAD DATA & MICROSTRUCTURE FEATURE ENGINEERING
# ======================================================================
def load_and_engineer_features(file_path="Asset_1_OB_snapshot_15000.csv", horizon=10):
    df = pd.read_csv(file_path)
    if 'Unnamed: 0' in df.columns:
        df.drop(columns=['Unnamed: 0'], inplace=True)

    feats = pd.DataFrame(index=df.index)
    mid_price = (df['bid_px1'] + df['ask_px1']) / 2.0
    feats['mid_price'] = mid_price

    for i in [1, 2, 3]:
        feats[f'spread_{i}'] = df[f'ask_px{i}'] - df[f'bid_px{i}']
        feats[f'rel_bid_px_{i}'] = df[f'bid_px{i}'] - mid_price
        feats[f'rel_ask_px_{i}'] = df[f'ask_px{i}'] - mid_price
        feats[f'obi_{i}'] = (df[f'bid_sz{i}'] - df[f'ask_sz{i}']) / (df[f'bid_sz{i}'] + df[f'ask_sz{i}'] + 1e-8)
        feats[f'log_sz_ratio_{i}'] = np.log((df[f'bid_sz{i}'] + 1) / (df[f'ask_sz{i}'] + 1))

    total_bid_sz = df['bid_sz1'] + df['bid_sz2'] + df['bid_sz3']
    total_ask_sz = df['ask_sz1'] + df['ask_sz2'] + df['ask_sz3']
    feats['total_obi'] = (total_bid_sz - total_ask_sz) / (total_bid_sz + total_ask_sz + 1e-8)

    micro_price = (df['bid_px1'] * df['ask_sz1'] + df['ask_px1'] * df['bid_sz1']) / (df['bid_sz1'] + df['ask_sz1'] + 1e-8)
    feats['micro_price_diff'] = micro_price - mid_price

    feats['return_lag1'] = mid_price.pct_change(1)
    feats['return_lag5'] = mid_price.pct_change(5)
    feats['rolling_vol_10'] = feats['return_lag1'].rolling(10).std()

    fwd_ret = (mid_price.shift(-horizon) - mid_price) / mid_price
    threshold = fwd_ret.std() * 0.5
```

```
43
44      target = pd.Series(1, index=df.index)
45      target[fwd_ret > threshold] = 2
46      target[fwd_ret < -threshold] = 0
47
48      feats['target'] = target
49      feats['fwd_return'] = fwd_ret
50
51      clean_df = feats.dropna().iloc[:-horizon].copy()
52      return clean_df, df
53
54  # ======================================================================
55  # 2. FEATURE SELECTION PIPELINE (IC & MULTICOLLINEARITY FILTER)
56  # ======================================================================
57  def select_top_features(df, train_idx, top_k=8, max_corr=0.85):
58      candidate_cols = [c for c in df.columns if c not in ['mid_price', 'target
            ', 'fwd_return', 'micro_price']]
59      train_df = df.iloc[:train_idx]
60
61      ic_scores = {col: abs(train_df[col].corr(train_df['fwd_return'], method='
            spearman')) for col in candidate_cols}
62      sorted_features = sorted(ic_scores.items(), key=lambda x: x[1], reverse=
            True)
63
64      selected = []
65      for feat, score in sorted_features:
66          if len(selected) >= top_k:
67              break
68          if not selected:
69              selected.append(feat)
70          else:
71              corrs = [abs(train_df[feat].corr(train_df[s])) for s in selected]
72              if max(corrs) < max_corr:
73                  selected.append(feat)
74
75      print(f"Selected Top {len(selected)} Stationary Features based on Rank IC
            :")
76      for f in selected:
77          print(f"  * {f:<18} | Rank IC: {ic_scores[f]:.4f}")
78
79      return selected
80
81  # ======================================================================
82  # 3. DATASET BUILDER & LSTM ALPHA MODEL
83  # ======================================================================
84  class LOBSequenceDataset(Dataset):
85      def __init__(self, X, y, seq_len=20):
86          self.seq_len = seq_len
87          self.X = torch.tensor(X, dtype=torch.float32)
88          self.y = torch.tensor(y, dtype=torch.long)
89
90      def __len__(self):
91          return len(self.X) - self.seq_len
92
93      def __getitem__(self, idx):
94          return self.X[idx : idx + self.seq_len], self.y[idx + self.seq_len]
95
96  class LOB_LSTM(nn.Module):
97      def __init__(self, input_dim, hidden_dim=64, num_layers=2, num_classes=3,
             dropout=0.2):
98          super(LOB_LSTM, self).__init__()
99          self.lstm = nn.LSTM(
```

```
            input_size=input_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0
        )
        self.fc = nn.Sequential(
            nn.Linear(hidden_dim, 32),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(32, num_classes)
        )

    def forward(self, x):
        lstm_out, _ = self.lstm(x)
        return self.fc(lstm_out[:, -1, :])

# ==========================================================================
# 4. INTEGRATED AVELLANEDA-STOIKOV BACKTESTER
# ==========================================================================
def run_integrated_backtest(clean_df, raw_df, train_size, model, scaler,
    selected_features, seq_len=20, device='cpu'):
    test_df = clean_df.iloc[train_size:].reset_index(drop=True)
    raw_test_df = raw_df.iloc[train_size:].reset_index(drop=True)

    mid_prices = test_df['mid_price'].values
    X_test_scaled = scaler.transform(test_df[selected_features].values)

    gamma = 0.05
    sigma = test_df['return_lag1'].std() * np.sqrt(100)
    k = 1.8
    beta = 0.35
    max_inv = 8

    inventory = 0
    cash = 0.0
    pnl_history = []
    inventory_history = []
    trade_pnls = []
    entry_prices = []

    model.eval()
    with torch.no_grad():
        for t in range(seq_len, len(test_df)):
            current_mid = mid_prices[t]
            best_bid = raw_test_df['bid_px1'].iloc[t]
            best_ask = raw_test_df['ask_px1'].iloc[t]

            seq_input = torch.tensor(X_test_scaled[t - seq_len : t], dtype=
                torch.float32).unsqueeze(0).to(device)
            logits = model(seq_input)
            probs = torch.softmax(logits, dim=1).cpu().numpy()[0]
            alpha_signal = probs[2] - probs[0]

            r = current_mid + (beta * alpha_signal) - (inventory * gamma * (
                sigma**2))
            spread_term = gamma * (sigma**2) + (2 / gamma) * np.log(1 + gamma
                 / k)
            optimal_spread = max(best_ask - best_bid, spread_term)

            ask_quote = r + (optimal_spread / 2.0)
            bid_quote = r - (optimal_spread / 2.0)
```

```
158
159             p_fill_bid = np.exp(-k * max(0.01, current_mid - bid_quote))
160             p_fill_ask = np.exp(-k * max(0.01, ask_quote - current_mid))
161
162             if inventory < max_inv and np.random.rand() < (p_fill_bid * 0.45)
                    :
163                 fill_price = min(bid_quote, best_bid)
164                 inventory += 1
165                 cash -= fill_price
166                 entry_prices.append(fill_price)
167
168             if inventory > -max_inv and np.random.rand() < (p_fill_ask *
                    0.45):
169                 fill_price = max(ask_quote, best_ask)
170                 inventory -= 1
171                 cash += fill_price
172                 if len(entry_prices) > 0:
173                     trade_pnl = fill_price - entry_prices.pop(0)
174                     trade_pnls.append(trade_pnl)
175
176             mtm = cash + (inventory * current_mid)
177             pnl_history.append(mtm)
178             inventory_history.append(inventory)
179
180     return np.array(pnl_history), np.array(inventory_history), np.array(
            trade_pnls)
181
182 # ==============================================================================
183 # MAIN EXECUTION PIPELINE
184 # ==============================================================================
185 if __name__ == "__main__":
186     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
187
188     clean_data, raw_data = load_and_engineer_features("
            Asset_1_OB_snapshot_15000.csv", horizon=10)
189     train_size = int(len(clean_data) * 0.8)
190
191     selected_cols = select_top_features(clean_data, train_idx=train_size,
            top_k=8)
192
193     X_all = clean_data[selected_cols].values
194     y_all = clean_data['target'].values
195
196     scaler = StandardScaler()
197     X_train_scaled = scaler.fit_transform(X_all[:train_size])
198     X_test_scaled = scaler.transform(X_all[train_size:])
199
200     seq_len = 20
201     train_ds = LOBSequenceDataset(X_train_scaled, y_all[:train_size], seq_len
            =seq_len)
202     val_ds = LOBSequenceDataset(X_test_scaled, y_all[train_size:], seq_len=
            seq_len)
203     train_loader = DataLoader(train_ds, batch_size=128, shuffle=True)
204
205     model = LOB_LSTM(input_dim=len(selected_cols), hidden_dim=64, num_layers
            =2,
206                      num_classes=3, dropout=0.2).to(device)
207     criterion = nn.CrossEntropyLoss()
208     optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e
            -5)
209
210     for epoch in range(1, 6):
```

```
        model.train()
        total_loss, correct, total = 0.0, 0, 0
        for X_b, y_b in train_loader:
            X_b, y_b = X_b.to(device), y_b.to(device)
            optimizer.zero_grad()
            out = model(X_b)
            loss = criterion(out, y_b)
            loss.backward()
            optimizer.step()
            total_loss += loss.item() * X_b.size(0)
            correct += (out.argmax(1) == y_b).sum().item()
            total += y_b.size(0)
        print(f"Epoch {epoch}/5 | Train Loss: {total_loss/total:.4f} | "
              f"Directional Accuracy: {correct/total*100:.2f}%")

    pnl_curve, inv_curve, trade_pnls = run_integrated_backtest(
        clean_df=clean_data, raw_df=raw_data, train_size=train_size,
        model=model, scaler=scaler, selected_features=selected_cols,
        seq_len=seq_len, device=device)

    final_pnl = pnl_curve[-1]
    peak = np.maximum.accumulate(pnl_curve)
    max_dd = np.max(peak - pnl_curve)
    win_rate = (np.sum(trade_pnls > 0) / (len(trade_pnls) + 1e-8)) * 100
    gross_profits = np.sum(trade_pnls[trade_pnls > 0])
    gross_losses = abs(np.sum(trade_pnls[trade_pnls < 0])) + 1e-8
    profit_factor = gross_profits / gross_losses

    print(f"Final Strategy PnL        : ${final_pnl:,.2f}")
    print(f"Maximum Drawdown          : ${max_dd:,.2f}")
    print(f"Calmar / Return-to-MDD    : {final_pnl / (max_dd + 1e-8):.2f}x")
    print(f"Closed Trade Win Rate     : {win_rate:.2f}%")
    print(f"Profit Factor             : {profit_factor:.2f}")
    print(f"Completed Round-Trips     : {len(trade_pnls)}")
    print(f"Max Absolute Inventory    : {np.max(np.abs(inv_curve))} units")
    print(f"Ending Inventory          : {inv_curve[-1]} units")
```

Listing A.1: Full pipeline: imports, feature engineering, feature selection, dataset/model definitions, integrated backtester, and main execution block.

CHAPTER B

Glossary of Terms

Term	Definition
Alpha signal	A model-derived forecast believed to have predictive power over future returns; here, the LSTM's $p_{\text{up}} - p_{\text{down}}$ output.
Ask (offer)	The price at which a seller is willing to sell.
Avellaneda–Stoikov model	A closed-form optimal market-making framework that derives inventory-skewed quotes and an optimal spread from risk aversion, volatility, and order-flow intensity parameters.
Bid	The price at which a buyer is willing to buy.
Calmar ratio	Total return divided by maximum drawdown; a measure of return per unit of worst-case pain.
Directional accuracy	The fraction of predictions where the model correctly classifies the direction (up/flat/down) of the future price move.
Drawdown	The decline in portfolio value from a previous peak.
Inventory	The net position (long or short) a market maker currently holds.
Level 1 (L1) data	Market data showing only the best bid, best ask, and their sizes.
Level 2 (L2) data	Market data showing multiple price levels of depth on both sides of the book.
Limit order book (LOB)	The live ledger of all outstanding buy and sell orders for an asset.
Liquidity	The ease with which an asset can be bought or sold without materially moving its price.
LSTM	Long Short-Term Memory; a recurrent neural network architecture designed to learn from ordered sequences.
Market maker	A participant who continuously quotes both a bid and an ask, earning the spread while managing inventory risk.
Mid-price	The average of the best bid and best ask.
Micro-price	A size-weighted refinement of the mid-price that leans toward whichever side of the book has less resting size.
Order book imbalance (OBI)	The normalised difference between resting bid and ask size at a given depth level.

Term	Definition
Poisson fill model	A probabilistic model in which the chance a resting quote is filled decays exponentially with its distance from the current price.
Profit factor	Gross profit from winning trades divided by gross loss from losing trades.
Rank Information Coefficient (Rank IC)	The Spearman rank correlation between a feature and a forward-looking return target, used to measure raw predictive power.
Reservation price	In the Avellaneda–Stoikov model, the price at which a market maker is indifferent between holding and not holding their current inventory.
Sharpe ratio	Mean return divided by the standard deviation of returns, typically annualised.
Slippage	The adverse difference between an expected trade price and the price actually achieved, often due to market impact.
Spread	The difference between the best ask and the best bid.
Tick	A single discrete update event in the order book (a new order, cancellation, or trade).
Volatility clustering	The empirical tendency for periods of high (or low) volatility to be followed by more periods of similarly high (or low) volatility.

References

1. Avellaneda, M., & Stoikov, S. (2008). High-frequency trading in a limit order book. *Quantitative Finance*, 8(3), 217–224.

2. Cont, R., Kukanov, A., & Stoikov, S. (2014). The price impact of order book events. *Journal of Financial Econometrics*, 12(1), 47–88.

3. Cartea, Á., Jaimungal, S., & Penalva, J. (2015). *Algorithmic and High-Frequency Trading*. Cambridge University Press.

4. Guéant, O. (2016). *The Financial Mathematics of Market Liquidity: From Optimal Execution to Market Making*. CRC Press.

5. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

6. Sirignano, J., & Cont, R. (2019). Universal features of price formation in financial markets: perspectives from deep learning. *Quantitative Finance*, 19(9), 1449–1459.

7. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

8. Paszke, A. et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32*.

9. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

10. McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.